

**TRAINWISE: A WEB AND MOBILE-BASED EMPLOYEE TRAINING
NEEDS ASSESSMENT AND RECOMMENDATION SYSTEM USING
XGBOOST AND NATURAL LANGUAGE PROCESSING**

A Capstone Project Presented to the Faculty, College of Computer Studies
Laguna State Polytechnic University
Los Baños Campus
Los Baños, Laguna

In partial fulfillment of the requirement for the degree
Bachelor of Science in Information Technology
Specialized in Web and Mobile Application Development

BRENDERICK I. AMPARO JR.
GIAN CARLO I. MARANAN
KLYDE CISCO T. CANAPI
SEAN JOHN R. LOOC

Alejandro V. Matute Jr.

ACKNOWLEDGMENT

In this section, the researchers must acknowledge all the people to help them during the conduct of their study. Hence, they can also acknowledge the organization that lends them data or funding support. Acknowledging the university is optional, however, it is suggested to give appreciation to it.

DEDICATION

The researchers can give dedication to their family and love ones or include those who they will be dedicating the project.

CHAPTER I

INTRODUCTION

PROJECT CONTEXT

Individuals have different understandings of success. Success in reality is about the humans that keep moving forward: no matter how big the obstacle is, they continue to strive for what's best in order to better themselves, to be innovative, and to sustain the coming days. The backbone of an institution is basically the employees who work. Their continuous improvements and natural adaptiveness are the key impacts of the production and challenge of the organization.

Because of globalization and the improvement of technology, the industry is evolving, and organizations need to adapt to its fast-paced changes and have a challenger mindset in their own personal sectors. Challenges and problems occur, making the organizations feel pressured, but it is needed so that they are not only ready but also perfectly adapting to evolution and conditioned to meet new obstacles in the way. With the competitiveness of the world, organizations should develop the minds of their employees to gain the success that everyone is yearning for. (Davenport and Ronanki, 2018)

Training is one of the principal ways that organizations promote this type of growth. It builds knowledge, sharpens skills, and outlines employees' career paths. "A good training program prepares workers for the job today and the job tomorrow" (Noe 2020). The problem is that training works when the organization

actually knows what its people need. Many companies still find it difficult to figure out which skills are lacking and which type of program will close those gaps. As Noe (2020) explained, training programs only deliver results when they are tied to both organizational goals and the real development needs of employees.

With the training needs assessment of the employees involved, we can say that it is very fundamental to improve, but in the coming years, most of the organization will stay in the traditional ways. In other words, these past methodologies are no longer sufficient for today's organizational demands; they are limited in scope and rely entirely on human judgment rather than systematic data analysis. Some examples include survey questionnaires for employees, one-on-one interviews, reviewing the specific performance of an employee, and the personal suggestions of the higher-ups. Now even after applying these methods to real-life experiences, the issues begin to surface. Most of the time, the results are very inconsistent, and specifically, the analysis is not based entirely on factual data but based merely on one's opinion. With continuous use of these past methods, they always fail to scan those very essential valuable records, especially in those large companies in which the information is so overloaded that it is very hard to keep an eye on every small detail without the use of automated machines.

These weaknesses cause issues for employees and the people who run the training programs. Basically, some employees are not receiving help at all; the reason is that they attend sessions that are really not helpful. This leads to employees not being interested and not improving much. On the other hand, the

people in charge have to deal with a lot of data that is hard to go through quickly. This makes it take longer to make decisions and slows down the process. The training programs are not used well. Employees do not get to develop their skills. As a company gets bigger, it becomes harder to manage employee information by hand. The problems with this way of doing things become even clearer. The people who run the company can see that it is not working well, as Salas et al. (2018) noted.

This shows that there is a difference between how companies are handling training now and what they really need to do to help their employees develop. Companies need a system that can do more than just gather information; they need something that can look at the information and figure out what it means and then use that to make something. The problem is not getting the information; it is doing something with the training data. Without the systems, companies have a lot of data from training that they cannot use fully. This underscores the need for organizations to adopt intelligent, technology-driven approaches to training needs assessment, as stated by Provost and Fawcett (2019).

The rapid advancement of technology has fundamentally changed how organizations manage information and support their workforce. Modern institutions have recognized that digital tools are no longer optional but essential for handling large volumes of data, facilitating communication across departments, and enabling faster, more informed decision-making. As Pressman and Maxim (2020) noted, web-based systems are particularly effective in

organizational settings because they can accommodate large numbers of concurrent users while remaining accessible across devices and locations — making them well-suited for institution-wide processes such as employee training management.

Alongside web-based platforms, artificial intelligence and machine learning have emerged as powerful tools for transforming raw organizational data into actionable insights. Machine learning enables systems to analyze large datasets, detect patterns, and generate predictions without requiring manual intervention at every step. Among the available algorithms, XGBoost — Extreme Gradient Boosting — is especially well-regarded for its accuracy and efficiency with structured tabular data. Developed by Chen and Guestrin (2016), XGBoost applies gradient boosting to iteratively correct prediction errors across successive decision trees, resulting in highly reliable classification performance. Zhang and Haghani (2019) further demonstrated its effectiveness in complex, real-world prediction tasks, showing improvements of approximately 15–20% over standard baseline models in multi-class classification scenarios.

However, numerical and structured data alone do not capture the full picture of employee training needs. Employees frequently express their development needs, concerns, and experiences through open-ended survey responses, comments, and self-assessments — forms of unstructured text that cannot be meaningfully quantified through conventional analysis. Natural Language Processing (NLP) addresses this gap by enabling systems to interpret human language, extract meaning from text, and identify patterns that would

otherwise go undetected. As Jurafsky and Martin (2023) explained, NLP allows systems to understand not just the words people use, but the deeper communicative intent behind them — making it possible for organizations to gain a more complete and accurate understanding of what their employees actually need.

Despite the availability of these technologies, many organizations have yet to integrate them meaningfully into their training needs assessment processes. Most existing systems are limited to data collection and storage, offering little in the way of intelligent analysis or automated recommendations. As a result, organizations accumulate large volumes of training-related data that remain underutilized, while decision-makers continue to rely on manual review and subjective judgment. Davenport and Ronanki (2018) emphasized that organizations must move beyond passive data collection toward systems that actively analyze information and support evidence-based decisions — a shift that is particularly critical in the context of employee development.

To address these limitations, this study presents TrainWise: a Web and Mobile-Based Employee Training Needs Assessment and Recommendation System using XGBoost and Natural Language Processing. TrainWise integrates data collection, intelligent analysis, and personalized training recommendations into a single platform accessible to both employees and administrators. The system applies XGBoost to analyze structured employee data — including department, designation, years of service, and training history — to identify competency gaps and predict appropriate training programs. Simultaneously, it

uses NLP, specifically Sentence-BERT (SBERT), to process employees' free-text responses and match their expressed training needs with relevant program descriptions based on semantic similarity rather than keyword matching alone.

By combining these two approaches, TrainWise enables organizations to make training decisions that are grounded in actual employee data rather than assumption or informal observation. The system reduces the time and manual effort required to assess training needs at scale, supports administrators with evidence-based reporting, and provides employees with personalized development recommendations aligned with both their individual profiles and organizational goals. As Provost and Fawcett (2019) argued, the organizations that are best positioned to remain competitive are those that transform the data they collect into meaningful, decision-ready intelligence — and that is precisely what TrainWise is designed to do.

Objectives of the Study

The general objective of this study is to design a proposed recommendation component based on XGBoost and Natural Language Processing that supports the identification of competency gaps and helps to generate training recommendations grounded in actual data to develop and improve TrainWise, a web-based employee training needs assessment and recommendation system.

To reach this, the study specifically aims to:

1. Develop a predictive recommendation model that uses XGBoost and Natural Language Processing (NLP) to go through employee data, surface possible competency gaps, and figure out the appropriate training needs based on both structured and unstructured inputs.
2. Design and develop a web-based application by integrating the recommendation model with automated TNA assessment, reporting, and recommendation functions.
3. Test and evaluate the existing and enhanced system using the Technology Acceptance Model (TAM) and ISO/IEC 25010 Software Quality Model in terms of usability, functionality, performance efficiency, and user acceptance.

Project Purpose

The purpose of this study is to provide organizations with a more systematic, data-driven approach to identifying the training needs of their employees. In many institutions, training programs are designed and implemented without a thorough understanding of what employees actually need to develop — resulting in misaligned programs, wasted resources, and missed opportunities for professional growth. Traditional training needs assessment methods, which rely on surveys, interviews, and manual evaluation, are often inconsistent and lack the analytical depth required to generate reliable, personalized recommendations at scale. This study addresses that gap by

developing and enhancing TrainWise, a web and mobile-based platform that uses XGBoost and Natural Language Processing to automate and improve the training needs assessment process.

TrainWise is designed to identify competency gaps by analyzing both structured employee data—such as department, designation, years of service, and training history—and unstructured text inputs, such as employees' self-assessments and desired training descriptions. By applying XGBoost to structured records and Sentence-BERT to textual inputs, the system is able to generate personalized training recommendations that are grounded in actual employee data rather than administrative assumptions. This integrated approach replaces time-consuming manual review with an evidence-based, automated process that is both faster and more consistent, supporting administrators and HR personnel in making well-informed training decisions across all departments.

The significance of this study extends to multiple stakeholders. For employees, TrainWise provides a clearer understanding of their strengths and areas for improvement, along with personalized recommendations that support meaningful professional development. For administrators and decision-makers, the system offers structured data, evaluation summaries, and monitoring tools that enable more efficient and objective planning of training programs. For organizations as a whole, TrainWise supports improved workforce competency, reduced training costs, and better alignment between employee development and institutional goals—contributing to long-term organizational sustainability and adaptability in a rapidly changing professional landscape.

This study also aligns with two of the United Nations Sustainable Development Goals. First, it supports SDG 4: Quality Education, which calls for inclusive, equitable, and lifelong learning opportunities for all. By ensuring that employees receive training that is relevant to their actual needs and professional contexts, TrainWise contributes to the goal of continuous, meaningful skill development within the workplace. Second, the study supports SDG 8: Decent Work and Economic Growth, which emphasizes the importance of productive employment and the development of a skilled, adaptable workforce. By improving the efficiency and accuracy of training needs assessment, TrainWise helps organizations build a more competent workforce—one that is better equipped to meet institutional demands, remain competitive, and contribute to sustained economic growth.

Employees. Faculty members and non-teaching personnel will benefit from having a clearer understanding of their strengths and areas for improvement through personalized, data-driven training recommendations. By receiving suggestions that are aligned with their actual competency profiles and professional contexts, employees can focus their development efforts on training programs that are genuinely relevant to their growth and career advancement within the institution.

Administrators and Decision Makers. HR administrators, unit heads, and college deans will benefit from access to organized assessment data, evaluation summaries, and monitoring tools that support more informed and efficient planning of training programs. Rather than relying on manual review or

informal observation, decision-makers can use TrainWise to ensure that the training programs they approve and implement are directly aligned with both employee development needs and the broader goals of the organization.

Organizations. Institutions such as Laguna State Polytechnic University will benefit from improved workforce competency, more efficient training management processes, and better overall organizational performance. By integrating intelligent data analysis into their training needs assessment workflows, organizations can respond more effectively to evolving institutional demands and maintain a competitive, well-prepared workforce.

Clients and Stakeholders. Although not direct users of the system, external clients and institutional stakeholders will indirectly benefit from the improved skills and performance of employees who receive more targeted and effective training. Better-trained personnel contribute to higher quality service delivery and stronger institutional outcomes.

Future Researchers. This study can serve as a reference and foundation for future research in intelligent systems, machine learning applications, and recommendation technologies within human resource and training management contexts — particularly studies involving XGBoost, Natural Language Processing, and Sentence-BERT for competency gap analysis and personalized recommendation generation.

The significance of this study lies in its potential to address a persistent gap in how organizations manage employee training and development. By integrating machine learning and natural language processing into a single, accessible platform, TrainWise offers a more systematic and evidence-based alternative to the manual, inconsistent methods that many institutions currently rely on. The system not only streamlines the training needs assessment process but also supports more informed decision-making at every level—from individual employees identifying their own development needs to administrators planning training programs to organizational leaders monitoring workforce competency across departments.

Beyond its immediate institutional application, this study demonstrates that intelligent recommendation technologies can be practically applied in an academic and human resource context, contributing to the growing body of knowledge on data-driven workforce development. TrainWise is significant not simply because it automates a process, but because it transforms how organizations understand and respond to the development needs of their people—making training more relevant, more efficient, and more aligned with long-term organizational goals.

Scope and Limitation of the Study

The study aims at developing and enhancing a web-based employee training needs assessment and recommendation system, called TrainWise, to

help organizations identify employee training needs, manage assessment records, monitor evaluation results, and support employee development planning. The present system provides basic functionalities like user registration, profile management, submission of training needs assessment, creation of Individual Development Plan, evaluation, reporting, notification management and document generation.

In the enhancement phase, the system will be designed to include a proposal for an intelligent recommendation engine which will use XGBoost and Sentence-BERT (SBERT) to assist in generating more personalized training recommendations. Structured data like employee and assessment data is processed using XGBoost and textual data like desired training, comments and other user responses is processed using SBERT. At the current stage, the machine learning component is treated as a proposed enhancement and will serve as the basis for future integration with the existing TrainWise platform. The following are the included modules:

Account Management Module

User Sign Up and Login. This system provides user accounts to select from Faculty Member, Non-Teaching Personnel, System Administrator (HR Admin) and College Dean. Users can log in with their registered credentials to use system features. This module ensures that certain functions are only available to users with the correct role. But new users are subject to approval by the administrator and may be delayed in access.

Profile Management Module

Profile Completeness Post registration, users are required to fill their profiles by providing information such as educational background, specialization, designation, department and years of experience. The system has a profile completion indicator to help users complete required fields. This ensures sufficient information is available for accurate analysis and recommendations. On the contrary, incorrect or missing information may affect the quality of the output of the system.

HR Admin Management Module

User Management and Dashboard. HR Admin Dashboard The HR Admin dashboard is the main hub of the system. It displays important information such as total users, submissions, and deadlines. This allows administrators to approve or deny user registration, manage users' accounts and monitor system activity. This module also includes a deadline setting for training needs assessment submissions. However, the system relies on continuous administrative oversight to keep the data updated and correct.

Assessment and Submission Module

Training Needs Assessment Forms. Users may fill out structured assessment forms by entering training history, attending seminars and desired training programs. The system collects structured data and free-form text input such as comments and suggestions. These data are analyzed to identify

competency gaps and training needs. However, the accuracy of results depends on the honesty of the user and the completeness of the information submitted.

Recommendation System Module

Training recommendation. The current TrainWise system has a recommendation section to display training related output to the user. This module, as the current version, is the space to display and review the recommendation results. The proposed enhancement will involve connecting the module to a Python-based machine learning recommendation engine that will provide more personalized training recommendations based on user profile information, submitted training needs assessment data, and desired training inputs.

The proposed recommendation engine will use XGBoost to analyze structured data such as department, role, employee type, assessment responses, and training history. Sentence-BERT (SBERT) will be used to process textual inputs and compare the meaning of the user's desired training with available training program descriptions. Through this enhancement, the system is expected to recommend training programs based on both structured employee information and semantic textual similarity. However, the recommendation output would still be subject to the completeness of the data provided by the user and the training program records available in the system.

Individual Development Plan (IDP) Module

Creation and Submission of IDP. Users can create and submit Individual Development Plans which identify their goals, competencies and areas for improvement. The system provides an IDP form that users can create, edit and submit as a resource for professional development. These plans can be viewed and tracked by administrators. However, these system-generated plans may not entirely replace expert evaluation and guidance. Evaluation Module (Department-wise)Assessment forms. The system contains evaluation forms administered by department level administrators such as College Deans.

Evaluation Module (By Department)

Evaluation Forms. The system includes evaluation forms managed by department-level administrators such as College Deans. These forms allow evaluators to assess employee performance and training outcomes. The module supports filtering by department, status, and employee type. However, evaluation results may still be influenced by subjective judgment.

Reporting and Monitoring Module

Submission Tracking and Reports. The system allows administrators to track assessment submissions, IDP forms, and evaluation results. It provides filtering and search functionalities for efficient data retrieval. This helps organizations monitor employee progress and training effectiveness. However,

reporting is limited to system-generated data and does not include external analytics tools.

General Limitations

The system is a generic solution for organizations and may require adaptations according to the institutional structure and training needs. TrainWise is a web and mobile-based platform and requires stable internet connectivity and basic digital literacy from the users.

The system depends on the accuracy and completeness of the data provided by the user, which directly affects the reliability of the predictions and recommendations. In addition, the system does not support real-time model retraining and it does not have integration with external training platforms, although it uses XGBoost and Sentence-BERT (SBERT).

In addition, the system is designed to assess training needs and recommend training programs, but does not require training participation and does not track long-term employee performance. The current version does not have advanced features like adaptive learning, AI feedback loops, and external certification integration.

CHAPTER II

THEORETICAL FRAMEWORK OF THE PROJECT

REVIEW OF RELATED LITERATURE, STUDIES, AND SYSTEMS

This chapter presents a review of literature, studies, and existing systems relevant to TrainWise: A Web and Mobile-Based Employee Training Needs Assessment and Recommendation System Using XGBoost and Natural Language Processing. The review covers five key areas that form the theoretical and technical foundation of the study: training needs assessment, machine learning, Natural Language Processing, recommendation systems, and web and mobile platforms. Each area is examined to establish the current state of knowledge, identify the limitations of existing approaches, and justify the need for an integrated, data-driven system such as TrainWise. The review draws on works by Noe (2020), Salas et al. (2018), Chen and Guestin (2016), Jurafsky and Martin (2023), Reimers and Gurevych (2019), Ricci et al. (2022), Pressman and Maxim (2020), and other relevant authors to situate the study within established research.

MANUAL TRAINING NEEDS ASSESSMENT

Training Needs Assessment (TNA) is a systematic process that organizations use to identify gaps between employees' current competencies and the skills required to meet organizational goals. When conducted effectively, TNA enables institutions to design targeted training programs that address real

development needs rather than general or assumed ones. As Noe (2020) emphasized, the success of any training program depends fundamentally on the quality of the needs assessment that precedes it—programs that are not grounded in actual employee data tend to produce limited results regardless of how well they are designed or delivered.

Traditional TNA methods typically rely on surveys, one-on-one interviews, supervisor observations, and manual performance reviews. While these approaches are straightforward to implement, they carry significant limitations. Noe (2020) noted that manual assessment methods are prone to subjectivity, as findings are shaped largely by the perspectives of individual evaluators rather than objective data. Salas et al. (2018) further found that non-standardized evaluation methods produce inconsistency rates of up to 40% in training needs identification, meaning that the same employee population may receive substantially different assessments depending on who conducts the review and how.

These limitations become more pronounced as organizations grow in size and complexity. Managing large volumes of employee records manually is slow, error-prone, and difficult to scale. Integrating different data sources—such as performance evaluations, training histories, and competency assessments—into a single coherent analysis is particularly challenging without automated tools. As a result, important patterns in employee development needs may go undetected, leading to training programs that are misaligned with what employees actually require. Salas et al. (2018) and Noe (2020) both concluded that organizations

seeking to improve the accuracy and efficiency of their training decisions must move toward data-driven, automated approaches to training needs assessment.

MACHINE LEARNING

Machine learning is a branch of artificial intelligence that enables systems to learn from data, identify patterns, and generate predictions without being explicitly programmed for each scenario. Unlike rule-based systems that follow fixed instructions, machine learning models improve their performance as they process more data, making them well-suited for complex, large-scale analytical tasks that would be impractical to perform manually. As Provost and Fawcett (2019) explained, machine learning transforms raw data into actionable information by uncovering relationships and trends that are not immediately visible through conventional analysis — a capability that is particularly valuable in organizational environments where data accumulates rapidly across multiple sources.

In the context of employee training, machine learning can analyze structured records such as performance scores, competency assessments, and training histories to identify patterns that indicate where development gaps exist. It can also support predictive analysis, helping organizations anticipate future training needs before gaps become performance problems. By reducing reliance on individual judgment and subjective evaluation, machine learning enables more consistent and scalable decision-making across departments and employee groups.

However, the effectiveness of machine learning depends heavily on the quality and completeness of the data it is trained on. Incomplete or inaccurate records will produce unreliable predictions, and building a well-performing model requires careful data preparation, feature engineering, and hyperparameter tuning. Despite these challenges, machine learning remains one of the most powerful tools available for organizations that need to derive meaningful insights from large and complex datasets — and its application to training needs assessment represents a significant opportunity to improve the precision and relevance of employee development programs.

XGBOOST

XGBoost, or Extreme Gradient Boosting, is a machine learning algorithm widely recognized for its accuracy, computational efficiency, and scalability across a range of prediction and classification tasks. Developed by Chen and Guestrin (2016), XGBoost applies gradient boosting — a technique in which successive decision tree models are trained to correct the errors of their predecessors — to produce a final model that is significantly more accurate than any individual tree. Through regularization techniques and efficient handling of missing values, XGBoost achieves strong predictive performance while reducing the risk of overfitting, making it suitable for real-world datasets that are often incomplete or unevenly distributed.

Chen and Guestrin (2016) demonstrated that XGBoost achieved up to 97.7% classification accuracy on benchmark datasets, outperforming other

gradient boosting methods in both speed and prediction quality. Zhang and Haghani (2019) further validated its effectiveness in complex, multi-class classification tasks, reporting accuracy improvements of approximately 15–20% over standard baseline models. These results established XGBoost as a preferred algorithm in data science and machine learning applications where high accuracy and processing efficiency are required simultaneously.

In the context of TrainWise, XGBoost is proposed to analyze structured employee data — including department, designation, specialization, years of service, teaching status, and assessment responses — to identify patterns associated with specific training needs. By processing these features, the model can detect which combinations of employee characteristics are most commonly associated with particular competency gaps, enabling the system to generate training recommendations that are grounded in data patterns rather than subjective evaluation. Effective implementation requires careful data preparation, appropriate feature encoding, and parameter tuning to ensure that the model generalizes well across different employee profiles and departments.

NATURAL LANGUAGE PROCESSING (NLP)

Natural Language Processing (NLP) is a field of artificial intelligence concerned with enabling computers to understand, interpret, and generate human language. In organizational settings, NLP is particularly valuable for analyzing unstructured text data — such as employee survey responses, open-ended assessment answers, and self-reported training preferences — that

cannot be meaningfully quantified through numerical analysis alone. As Jurafsky and Martin (2023) explained, NLP allows systems to extract meaning from text by analyzing not just the words used but the intent and context behind them, enabling a deeper and more accurate interpretation of what individuals are communicating.

In training needs assessment, employees frequently describe their development needs in their own words through comments, feedback forms, and open-ended survey questions. This type of qualitative input contains valuable insights that structured questionnaires may fail to capture — insights about specific concerns, preferred learning areas, and self-perceived gaps that are difficult to express through multiple-choice formats. Without automated text analysis, however, this information often goes unprocessed due to the volume and variability of responses. NLP addresses this challenge by systematically analyzing textual inputs, identifying recurring themes, and extracting relevant information that can directly inform training decisions.

Despite its capabilities, NLP is not without limitations. Language is inherently context-dependent and culturally variable, which means that models must be carefully trained on representative data to perform reliably across different communication styles and professional vocabularies. Additionally, NLP systems require regular updates to remain effective as language evolves. When implemented correctly, however, NLP significantly enhances a system's ability to derive actionable insights from unstructured data, making it an essential

complement to the structured data analysis provided by algorithms such as XGBoost.

SENTENCE-BERT (SBERT) IN NATURAL LANGUAGE PROCESSING

Sentence-BERT, commonly referred to as SBERT, is a specialized NLP model developed by Reimers and Gurevych (2019) as an extension of the original BERT architecture—Bidirectional Encoder Representations from Transformers. While BERT processes text by analyzing the contextual relationships between individual words in both directions within a sentence, its original design was not optimized for efficiently generating fixed-length representations of entire sentences. SBERT addresses this limitation by using a Siamese network structure to produce sentence-level embeddings—numerical vector representations that capture the overall semantic meaning of a sentence—enabling fast and accurate comparison between different pieces of text.

The primary advantage of SBERT over conventional keyword-matching approaches is its ability to measure semantic similarity rather than surface-level word overlap. When two sentences express the same idea using different vocabulary, SBERT recognizes their similarity based on meaning rather than exact wording. Reimers and Gurevych (2019) demonstrated that SBERT achieved up to 89.6% Spearman correlation on the Semantic Textual Similarity benchmark and operates approximately 80 times faster than standard BERT for similarity tasks—making it practical for real-time recommendation applications.

In TrainWise, SBERT is proposed to process the desired training text submitted by employees and compare it semantically with the descriptions of available training programs stored in the system database. For example, if an employee submits that they wish to improve their skills in protecting organizational data, SBERT can identify training programs related to cybersecurity, information security, or data privacy—even if the employee's exact words do not appear in those program descriptions. This semantic matching capability makes the recommendation process more flexible and contextually aware than keyword-based search, ensuring that employees receive suggestions that genuinely align with their expressed needs rather than only those that happen to share identical terminology.

WEB, MOBILE, AND PROGRESSIVE WEB APPLICATION (PWA) SYSTEMS

Web and mobile platforms have become foundational infrastructure for modern organizational systems, enabling institutions to manage large volumes of data, facilitate communication across departments, and provide users with access to system functions from any location and device. As Pressman and Maxim (2020) noted, web-based systems are especially effective in multi-user organizational environments because they can accommodate concurrent access from many users while maintaining data integrity and system performance—characteristics that are essential for institution-wide processes such as training needs assessment.

Sommerville (2019) further observed that modern software systems benefit significantly from responsive, well-integrated interfaces that allow users to interact with data in real time without performance degradation. In practice, this means that employees can submit assessments, view recommendations, and track their development plans from desktops, laptops, or smartphones through a single web-based platform—eliminating the need for separate device-specific applications. This accessibility is particularly important in institutional settings where users vary widely in their technical proficiency and available devices.

Progressive Web Applications (PWAs) extend the capabilities of standard web platforms by combining web accessibility with mobile-like features such as offline functionality, fast loading, and device-responsive design. Unlike native mobile applications, PWAs do not require installation from an app store and can be updated centrally without user intervention, reducing both development costs and maintenance overhead. While TrainWise is implemented as a responsive web application rather than a full PWA, the responsive design principles underlying PWA development informed the system's interface architecture, ensuring that it functions reliably across a range of screen sizes and browser environments.

RECOMMENDATION SYSTEMS IN TRAINING AND DEVELOPMENT

Recommendation systems are computational tools designed to suggest relevant items or actions to users based on their data, preferences, and behavioral patterns. Originally developed for e-commerce and entertainment

platforms, recommendation systems have increasingly been applied to educational and professional development contexts, where they help match learners with resources, courses, or training programs that are most relevant to their individual needs and goals. As Ricci et al. (2022) explained, personalized recommendation systems improve user experience and decision-making efficiency by filtering large sets of options down to a manageable, contextually appropriate selection—a function that is particularly valuable when the number of available training programs exceeds what any individual employee can meaningfully evaluate independently.

In the context of employee development, recommendation systems analyze employee profiles, assessment records, competency evaluations, and expressed preferences to identify which training programs are most likely to address each individual's development gaps. Aggarwal (2019) found that personalized recommendation algorithms improved user engagement by approximately 25% and supported up to 30% gains in decision-making efficiency compared to manual selection processes—demonstrating the practical impact of intelligent recommendations on organizational outcomes.

However, the effectiveness of recommendation systems depends on the quality and completeness of the data on which they operate. Systems that rely on outdated, incomplete, or inaccurate records risk generating irrelevant or misleading suggestions, which can erode user trust and reduce adoption. Additionally, systems that focus solely on generating recommendations without incorporating evaluation and monitoring functions cannot fully support the

ongoing development needs of employees. TrainWise addresses these limitations by integrating the recommendation function within a broader platform that also handles assessment submission, evaluation, IDP creation, and administrative reporting — ensuring that recommendations are generated from current, validated data and can be reviewed, tracked, and refined over time.

SYNTHESIS AND RELEVANCE OF THE STUDY

The literature reviewed in this chapter consistently points to a clear and growing need for intelligent, data-driven approaches to employee training needs assessment. Across the works examined, a common pattern emerges: while organizations recognize the importance of identifying and addressing employee competency gaps, the methods they currently rely on are insufficient to meet the demands of modern institutional environments. Manual assessment processes — including surveys, interviews, and supervisor evaluations — are widely documented as inconsistent, time-consuming, and prone to subjective bias (Noe, 2020; Salas et al., 2018). As organizations grow in size and complexity, these limitations become increasingly costly, resulting in misaligned training programs, wasted resources, and missed opportunities for workforce development.

The reviewed literature also demonstrates that the technologies required to address these limitations are well-established and proven. Machine learning, particularly XGBoost, has been shown to deliver accurate and scalable classification performance on structured organizational data, enabling systems to

detect patterns in employee profiles and assessment records that would be difficult or impossible to identify through manual review (Chen & Guestrin, 2016; Zhang & Haghani, 2019). Natural Language Processing, and specifically Sentence-BERT, extends this capability to unstructured text by generating semantic embeddings that allow systems to interpret the meaning of employee-submitted responses and match them with relevant training programs — even when exact keywords do not align (Jurafsky & Martin, 2023; Reimers & Gurevych, 2019). Recommendation systems built on these technologies have been shown to improve decision-making efficiency and user engagement significantly compared to manual selection (Ricci et al., 2022; Aggarwal, 2019). Web and mobile platforms, meanwhile, provide the accessibility and scalability needed to deploy such systems across large, multi-role institutional environments (Pressman & Maxim, 2020; Sommerville, 2019).

Despite the availability of these technologies, a significant gap remains between what current systems offer and what institutions such as Laguna State Polytechnic University actually need. Most existing training management systems are limited to data collection and storage, offering little in the way of intelligent analysis or automated recommendation generation. Assessment, analysis, and recommendation functions are often handled by separate, disconnected tools—or not at all—leaving administrators without the integrated decision support required to manage training needs effectively at the departmental and institutional level. This fragmentation is precisely the gap that the reviewed

literature identifies as the central barrier to more effective workforce development.

TrainWise directly addresses this gap by combining training needs assessment, intelligent data analysis, and personalized recommendation generation into a single, integrated web and mobile platform. By applying XGBoost to structured employee data and Sentence-BERT to textual training inputs, the system is designed to produce more accurate, relevant, and personalized training recommendations than manual or disconnected approaches can provide. The integration of IDP creation, evaluation, reporting, and notification management within the same platform further ensures that training decisions are supported by complete, current, and role-appropriate data at every stage of the process. In doing so, TrainWise represents a practical application of the technologies and principles documented in the reviewed literature—translating established research findings into a functional system designed specifically for the institutional context of LSPU Los Baños Campus.

Table 1. Technology Gap Analysis Table

Authors	Title	Field	Method / Algorithm	Result
Noe (2020)	Employee Training and Development	Human Resource Development	Manual Training Needs Assessment	Emphasized that accurate training needs assessment improves training effectiveness, but relies on subjective evaluation. Studies indicate that structured TNA improved training effectiveness by

Authors	Title	Field	Method / Algorithm	Result
Salas et al. (2018)	The Science of Training and Development	Organizational Training	Surveys, Interviews, Evaluations	approximately 35% over ad hoc approaches, yet the manual process limits scalability. Found that traditional training methods are often inconsistent, time-consuming, and prone to bias. Research suggests up to 40% inconsistency rate in training needs identification when using non-standardized evaluation methods. Demonstrated that AI enhances data-driven decision-making and reduces reliance on manual processes. AI-driven systems achieved approximately 45% reduction in manual processing time in organizational decision-support tasks.
Davenport & Ronanki (2018)	Artificial Intelligence for the Real World	Artificial Intelligence	AI-Based Decision Systems	Achieved high accuracy, efficiency, and scalability in predictive modeling tasks. XGBoost reached up to 97.7% classification accuracy on benchmark datasets and outperformed other gradient boosting methods in speed and prediction performance.
Chen & Guestrin (2016)	XGBoost: A Scalable Tree Boosting System	Machine Learning	XGBoost Algorithm	

Authors	Title	Field	Method / Algorithm	Result
Zhang & Haghani (2019)	A Gradient Boosting Method for Prediction	Machine Learning	Boosting Algorithms (XGBoost)	Showed improved classification and prediction performance using large and complex datasets. Gradient boosting improved prediction accuracy by approximately 15–20% over standard baseline models in multi-class classification tasks.
Jurafsky & Martin (2023)	Speech and Language Processing	Natural Language Processing	NLP Techniques (Text Analysis)	Enabled extraction of meaningful insights from unstructured textual data, improving the analysis of human language. NLP-based text classification tasks achieved over 88% accuracy on benchmark language datasets.
Devlin et al. (2019)	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	Natural Language Processing	BERT Model	Improved contextual understanding of language through bidirectional processing, enhancing NLP performance. BERT achieved 93.5% accuracy on the GLUE benchmark, outperforming prior NLP methods by up to 7.7 percentage points.
Reimers & Gurevych (2019)	Sentence-BERT: Sentence Embeddings using Siamese	Natural Language Processing	Sentence-BERT (SBERT)	Enabled fast and accurate semantic similarity comparison, making it suitable for

Authors	Title	Field	Method / Algorithm	Result
	BERT-Networks			recommendation and matching systems. SBERT achieved up to 89.6% Spearman correlation on the STS benchmark and is approximately 80x faster than standard BERT for similarity tasks.
Ricci et al. (2022)	Recommender Systems Handbook	Recommendation Systems	Collaborative & Content-Based Filtering	Improved personalization and user experience through tailored recommendations. Collaborative filtering methods improved recommendation precision by 20–30% compared to non-personalized systems across multiple evaluation benchmarks.
Aggarwal (2019)	Recommender Systems: The Textbook	Data Mining	Recommendation Algorithms	Increased user engagement and supported more efficient decision-making. Recommendation algorithms demonstrated approximately 25% improvement in user engagement and up to 30% gain in decision-making efficiency over manual selection.

CONCEPTUAL FRAMEWORK

The conceptual framework of this study illustrates how the proposed system, TrainWise: A Web and Mobile-Based Employee Training Needs Assessment and Recommendation System using XGBoost and Natural Language Processing, operates by transforming user inputs into meaningful and data-driven outputs that support training and development within Laguna State Polytechnic University.

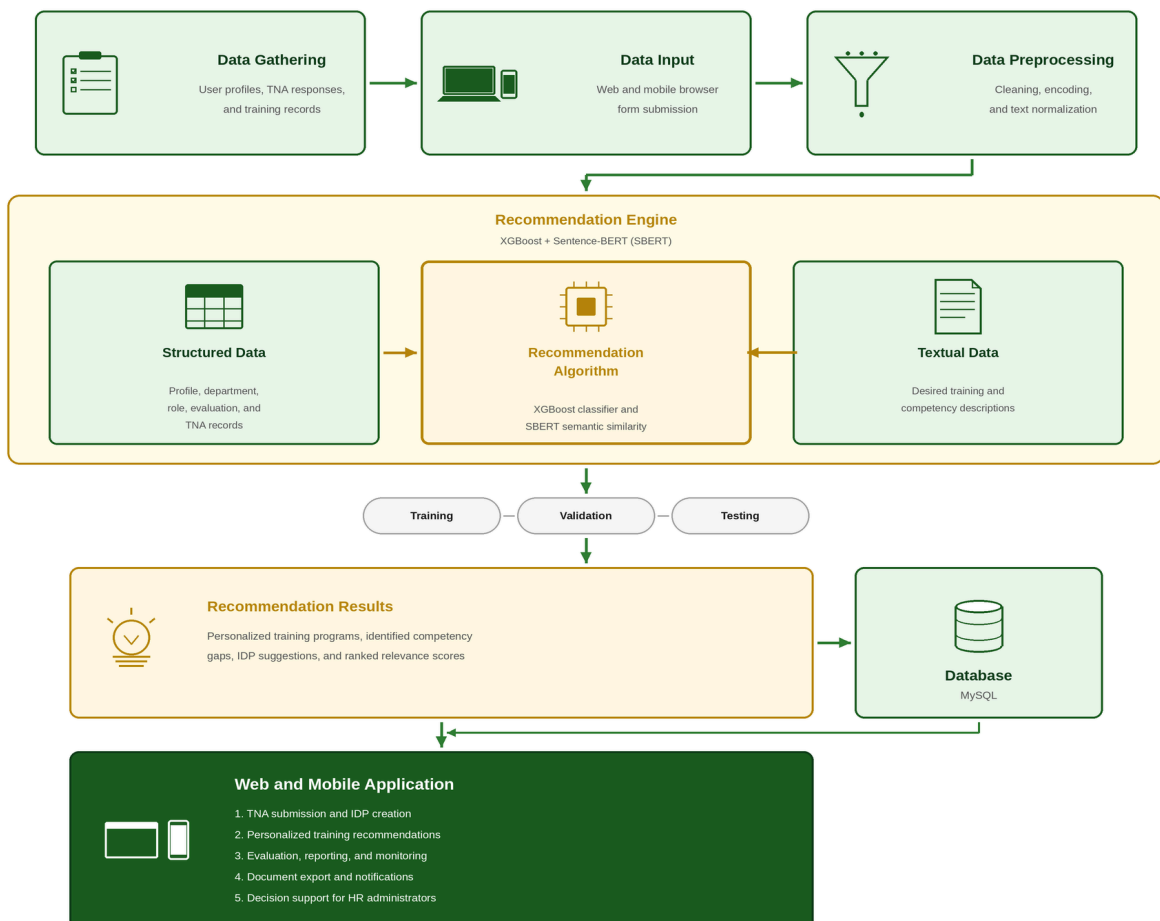


Figure 1. Conceptual Framework of the System

The framework follows a structured input-process-output flow. The process begins with Data Gathering, where user profiles, TNA responses, and training records are collected from employees. This information is submitted into the system through Data Input using the web and mobile browser. The collected information then undergoes Data Preprocessing, which includes cleaning, encoding, and text normalization to prepare both structured and textual data for analysis.

The preprocessed information is then handled by the Recommendation Engine, which combines XGBoost and Sentence-BERT (SBERT). Inside the engine, Structured Data — composed of profile, department, role, evaluation, and TNA records — and Textual Data — composed of desired training and competency descriptions — are processed by the Recommendation Algorithm, where the XGBoost classifier analyzes structured patterns and SBERT performs semantic similarity matching. The engine undergoes Training, Validation, and Testing to ensure the reliability and accuracy of the generated outputs.

The processed outputs are stored in the MySQL Database and presented as Recommendation Results, which include personalized training programs, identified competency gaps, IDP suggestions, and ranked relevance scores. These results are delivered to users through the Web and Mobile Application, which supports five core functions: (1) TNA submission and IDP creation, (2) personalized training recommendations, (3) evaluation, reporting, and monitoring, (4) document export and notifications, and (5) decision support for HR administrators.

Through this framework, TrainWise integrates data collection, intelligent analysis, and recommendation generation into a single platform, transforming the training needs assessment process from manual, opinion-based methods into an automated and evidence-driven approach.

CHAPTER III

METHODOLOGY

This chapter describes the techniques used to create and enhance TrainWise, a web and mobile solution that uses XGBoost and natural language processing to evaluate employee training needs and provide recommendations. The project design, system construction, architecture, database setup, testing procedures, and evaluation of the outcomes are all covered. To enable continuous planning, development, testing, and updates, the project employs the Web and Mobile Application Development track and the Agile Scrum methodology.

Project Design

The TrainWise project illustrates the connections between user requirements, system modules, database operations, and the proposed recommendation engine using concepts from system design. The existing system is a responsive web platform for evaluating the training needs of employees. Users can create reports, manage development programs, assess employees, provide training materials, and maintain training records. A machine learning-based recommendation engine will be incorporated in the following stage to offer tailored training recommendations.

The TrainWise system architecture is displayed in Figure 2. The Users Layer, Frontend Layer, Backend Services Layer, Machine Learning Engine Layer, Database Layer, and Deployment Layer make up the six main layers of the architecture. The primary operations of the system, including user interface, data processing, storage, suggestion creation, and report generation, are supported by these layers working together. The final data and documents generated by the system, such as customized training recommendations, competency gap insights, assessment reports, Individual Development Plans, evaluation summaries, monitoring reports, and exported PDF documents, are represented by the System Outputs section of the architecture.

The various users of the system are displayed in the **Users Layer**. These users include the HR or system administrators, college deans or evaluators, faculty members or employees, and non-teaching staff. The system is used by faculty and non-teaching staff to complete their profiles, submit training needs forms, select the training they wish to receive, and view their training outcomes. It is used by college deans or evaluators to monitor department submissions and evaluate employees. User accounts, deadlines, assessment records and reports, and system administration are all handled by HR and system administrators. Each user can only see the features and data required for their function thanks to this configuration.

The portion of the system that users interact with via a web browser is known as the **Frontend Layer**. Because it was created using HTML, CSS, and

JavaScript, it can be seen on desktop and mobile devices. Viewing dashboards, completing evaluation forms, checking profile pages, viewing results, reading reports, and utilizing additional features are all possible with the frontend. TrainWise doesn't require a separate mobile app because it is responsive, meaning it functions well on many screen sizes.

The **Backend Services Layer** is the main processing component of TrainWise. Its PHP-based application layer manages business logic, system workflows, database communication, report generation, and integration with the proposed Python-based recommendation engine. It encompasses the following modules:

Account Management Module. This module handles user registration, login authentication, account verification, and role-based access control. It ensures that administrators, evaluators, faculty, and non-teaching personnel can only access the features and data relevant to their designated role.

Profile Management Module. Users complete and update personal and professional information — including department, designation, educational attainment, specialization, and years of service — through this module. Profile data serves as one of the primary structured inputs used by the proposed recommendation engine when generating training suggestions.

Assessment and Submission Module. This module collects both structured responses and free-text inputs, such as desired training descriptions and comments, through the Training Needs Assessment form. All submitted data

are stored in the database and made available for future processing by the recommendation engine.

Recommendation Module. In the current system, this module displays the space where recommendation results will appear. In the proposed enhancement, it will connect to the Python-based machine learning engine to deliver personalized training recommendations derived from the user's profile and TNA submissions.

Individual Development Plan (IDP) Module. Users create, edit, and submit development plans that outline their training objectives, target competencies, and planned activities. Administrators and evaluators may access these plans to monitor staff progress and guide training decisions.

Evaluation Module. Authorized evaluators — such as College Deans — use this module to assess employee submissions at the department level, applying filters by department, status, and employee type to support organized performance review.

Reporting and Monitoring Module. Administrators access assessment submissions, IDP records, and evaluation summaries through this module, with filtering and search functions that support efficient data retrieval and institutional monitoring.

Notification and Deadline Management Module. This module allows administrators to set TNA submission deadlines and send automated reminders and status updates to users via integrated email notifications.

PDF Generation and Document Export Module. The system generates downloadable, printable documents — including TNA reports, IDP forms, and evaluation summaries — through this module, giving users and administrators formal copies of system-generated records.

The **Machine Learning Engine Layer** is the organized smart upgrade of TrainWise. This layer will be developed as a Python API that will connect with the existing PHP based core system. The suggested recommendation engine, once built, will get the evaluation data and the needed training text from the backend services layer. It will have multiple processes to handle the data, including data preprocessing, structured data analysis, text embedding, semantic analysis, prediction based on XGBoost, and similarity matching with Sentence-BERT.

The **XGBoost** component is designed to analyze structured data, including user profile information, department, role, assessment responses, and training-related records. Its objective is to recognize patterns in the data of the current system and suggest training programs that may be pertinent to the user. The SBERT component is designed to convert textual inputs into semantic representations and compare them with the descriptions of available training programs in order to process the user's desired training text. Once the machine

learning feature is completely implemented, TrainWise will be able to generate more meaningful recommendation outputs through this planned process.

Structured data, including user profile information, department, role, assessment responses, and training-related records, will be analyzed by the **XGBoost Model**. Its objective is to anticipate training programs that may be pertinent to the user by identifying patterns in the data of the existing system. This component fortifies the system by enabling it to generate recommendations based on structured employee information, rather than solely relying on manual selection.

The user's preferred training text will be processed by the **Sentence-BERT (SBERT) Model**. It will evaluate the meaning of the user's input with the descriptions of available training programs and convert textual input into semantic representations. This enables the system to comprehend user requirements even when the precise words do not correspond to the title of the training program. For example, the SBERT component can still match a user's desired training program to training programs related to cybersecurity, data protection, or network security based on meaning if the user inputs a training program related to improving system security skills.

MySQL is employed by the **Database Layer** to store and administer the system's data. It encompasses user profiles, assessment records, evaluation data, IDP records, notifications, preferences, deadlines, training program data,

and recommendation results. TrainWise's central storage is the database, which facilitates both the current main system and the proposed machine learning enhancement. It supplies the data required for the generation of documents, monitoring, recommendation processing, and reporting.

The **Email/Notification Service** facilitates communication between the system and its users. It is used to communicate reminders, deadlines, and status updates. This service is linked to the Notification and Deadline Management Module to keep users informed about key training needs assessment activities.

The **Deployment Layer** describes the environment in which the system and its components will operate. The PHP-based web system is served via the web hosting or server environment, whereas the proposed recommendation engine is run by the Python API service. Continuous monitoring is also offered to help with system maintenance, updates, and log checks. This layer assures that the core system and proposed machine learning service will function properly once deployed.

TrainWise produces final results called **System Outputs**. These include training recommendations, insights into competency gaps, assessment reports, Individual Development Plans, evaluation summaries, administrative monitoring reports, and exported PDF documents. These outputs help users and administrators identify training needs, organize staff development plans, and make data-driven training decisions.

In general, the TrainWise project design integrates the existing web-based training requirements assessment system with a proposed intelligent recommendation engine. The proposed XGBoost and SBERT components improve the system by generating more relevant and personalized training recommendations, while the existing system provides the foundation for user administration, assessment collection, IDP creation, evaluation, reporting, and documentation. TrainWise is transformed into a decision-support tool for the assessment of training requirements and the development of employees as a result of this design, which elevates it beyond the realm of basic information management.

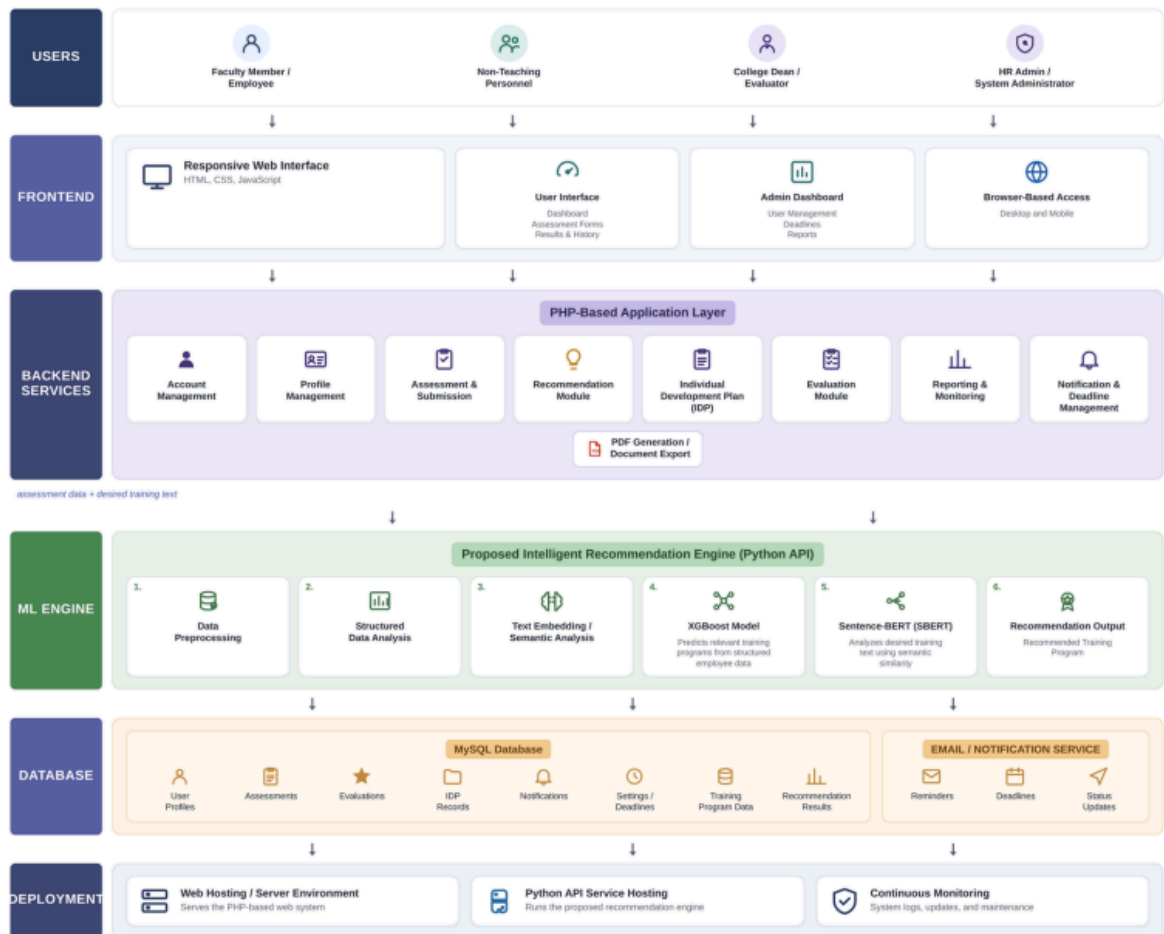


Figure 2. System Architecture

System Flowchart

Figure 3 presents the TrainWise system flowchart, which shows each step from accessing the system to generating a report. To begin, users open TrainWise in a web browser. New users need to register and wait for their account to be confirmed. After approval, they can log in and move forward.

The system checks the user's credentials after they log in. The user is taken back to the login procedure if the credentials are incorrect. The system determines the user's role and points them to the relevant dashboard if the credentials are legitimate. The flowchart divides the system functioning into three main role-based processes: HR Administrator or System Administrator, College Dean or Evaluator, and Faculty Member or Non-Teaching Personnel.

The system enables faculty members and non-teaching staff to access the user dashboard, fill out or update their profile, submit the Training Needs Assessment form, and enter the training they would want to receive. The MySQL database contains the submitted data. Selected assessment data and intended training inputs will be sent to the suggested Python Recommendation API in the improved TrainWise version. Sentence-BERT is used to analyze the needed training text and XGBoost is used to process structured employee data in the proposed machine learning service. After it is put into operation, the recommendation engine will produce training suggestion outputs and competency gap insights that the user may see.

The technology gives college deans and evaluators access to the evaluator dashboard, where they can analyze individual development plans and assessments that have been filed. In addition, evaluators have access to department reports, evaluation summaries, and personnel assessments. At the departmental level, this position aids in the monitoring and verification of training-related data.

The system offers administration features for HR administrators or system administrators, including user management, account approval or denial, deadline and notification setting, training program data management, assessment and evaluation monitoring, and administrative report generation. Additionally, the administrator can export PDF files for reporting and documentation needs.

The flowchart also shows the MySQL database as the central storage of the system. It stores and manages user profiles, submitted assessments, evaluation records, IDP records, notifications, training program data, and recommendation results. The system outputs include personalized training recommendations, competency gap insights, assessment reports, Individual Development Plans, evaluation summaries, administrative monitoring reports, and exported PDF documents.

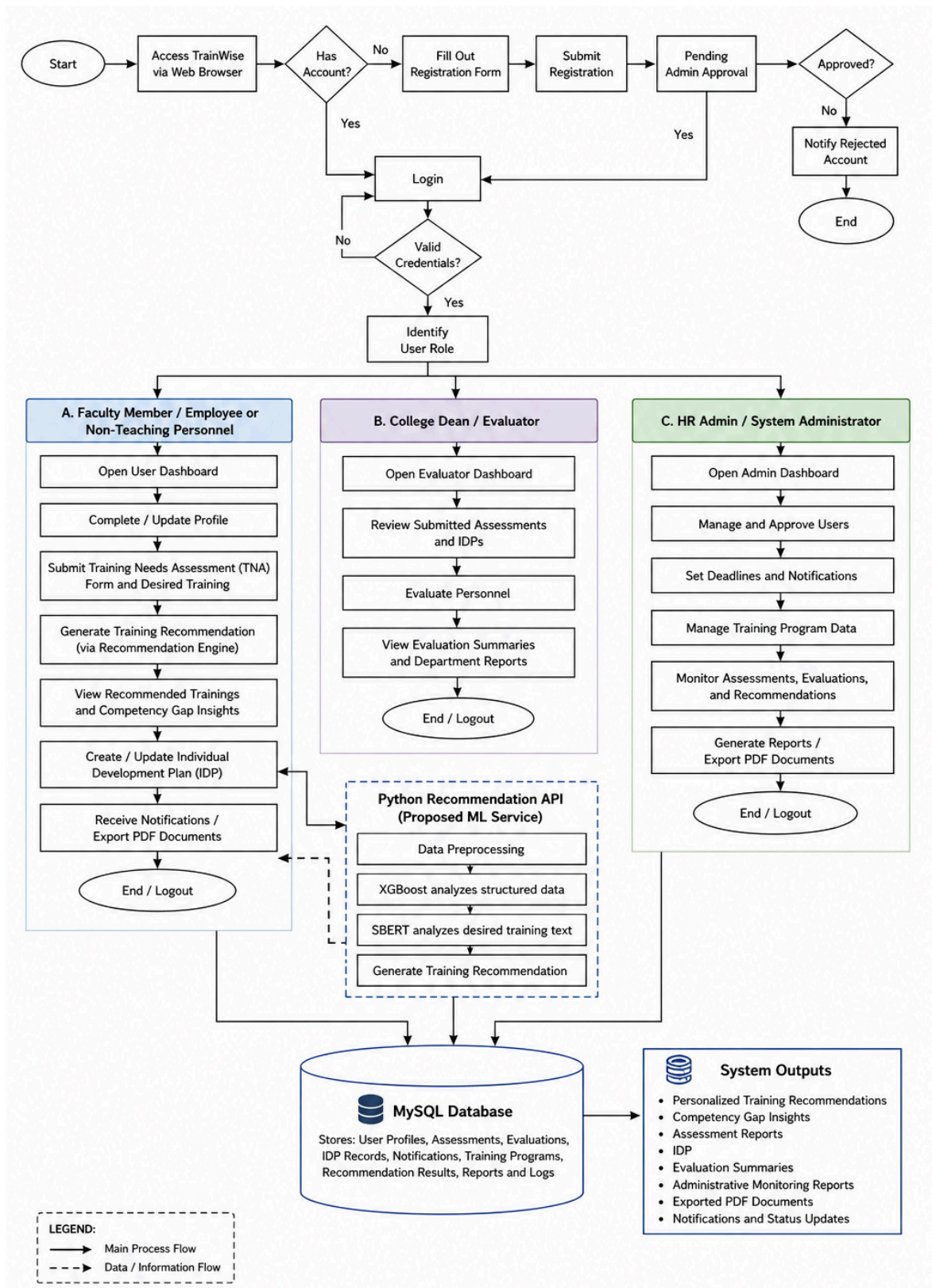


Figure 3. System Flowchart of TrainWise

Use Case Diagram

Figure 4 presents the updated use case diagram of TrainWise, which illustrates the interaction between the system actors and the major functions of the web-based employee training needs assessment and recommendation system. The diagram was designed to show the functional scope of the system in a simplified and role-based manner. Instead of presenting all internal processes in detail, the updated diagram focuses on the main functions that are directly accessed or triggered by each actor, making it more readable and suitable for inclusion in the capstone paper.

The use case diagram consists of four main actors: the **End User**, the **College Dean** or **Evaluator**, the **HR Admin** or **System Administrator**, and the **Recommendation Engine**. The End User represents both Faculty Members or Employees and Non-Teaching Personnel since these users perform similar actions within the system. The College Dean or Evaluator represents the authorized personnel responsible for reviewing submissions and evaluating employees within their assigned department. The HR Admin or System Administrator serves as the main system manager who controls user access, deadlines, training program records, reports, and notifications. The Recommendation Engine represents the proposed intelligent component that supports the generation of personalized training recommendations based on submitted assessment data and desired training inputs.

For the **End User**, the system provides essential functions such as registering an account, logging in, managing the profile, submitting the Training Needs Assessment (TNA) form with desired training, viewing training recommendations, creating or updating the Individual Development Plan (IDP), viewing notifications, and logging out. These use cases represent the primary activities of employees and non-teaching personnel when using TrainWise. Through these functions, users can provide the necessary information for training needs assessment, receive recommended training programs, and organize their development plans based on identified competency gaps.

The **College Dean or Evaluator** is given access to functions related to assessment review and personnel evaluation. These include reviewing submissions, evaluating personnel, viewing evaluation summaries, and viewing department reports. These use cases allow evaluators to monitor the training needs and evaluation results of employees under their department or area of responsibility. Through these functions, evaluators can support the validation of employee development needs and contribute to more organized training decisions.

The **HR Admin or System Administrator** manages the administrative side of TrainWise. The administrator can manage users, set deadlines, manage training programs, monitor reports, generate or export PDF documents, and send notifications. These functions support the overall operation of the system by ensuring that user accounts are properly maintained, assessment schedules are monitored, training program data are updated, reports are available, and users

are informed about important deadlines or system updates. This role is important in maintaining the accuracy, accessibility, and organization of training-related records within the system.

The **Recommendation Engine** is shown as a separate supporting actor because it represents the proposed intelligent recommendation component of TrainWise. Its main function is to support the generation of training recommendations based on information submitted by the user. The diagram shows an `<<include>>` relationship from “Submit TNA Form / Desired Training” to “Generate Training Recommendation,” indicating that the recommendation process will be triggered once the proposed machine learning feature is integrated with the system.

Although the proposed enhancement is designed to use XGBoost and Sentence-BERT (SBERT), these internal machine learning processes are not shown separately in the updated diagram to avoid overcrowding the figure. Instead, they are represented under the single use case “Generate Training Recommendation (Proposed ML Output),” while the detailed explanation of the machine learning process is discussed in the system architecture and recommendation engine sections.

Overall, the use case diagram shows how TrainWise supports multiple role-based operations within one integrated system. It demonstrates how end users can submit training needs and view recommendations, how evaluators can review and assess personnel, how administrators can manage system

operations and reports, and how the Recommendation Engine assists in producing personalized training recommendations. Through this structure, TrainWise functions not only as a training needs assessment platform but also as a decision-support tool for employee development and training management.

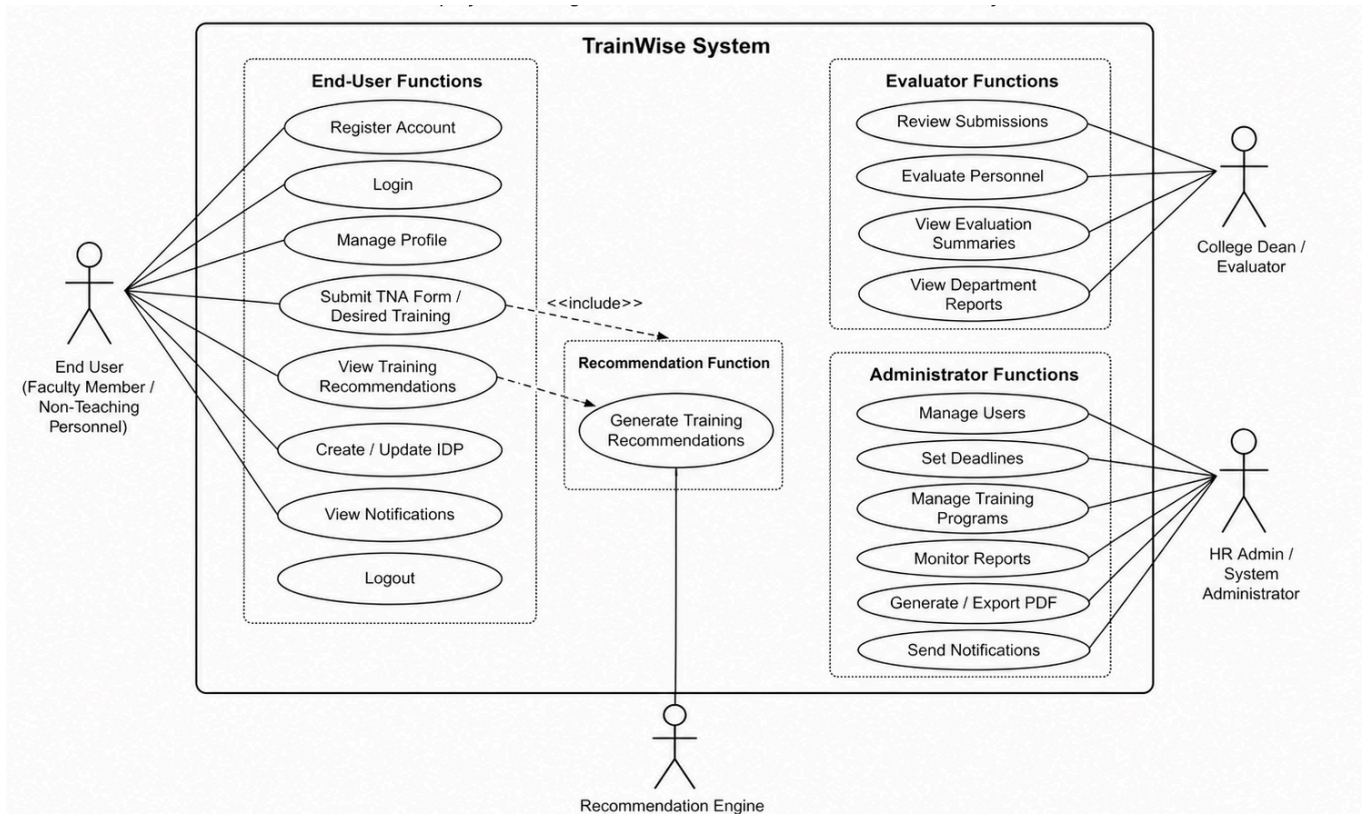


Figure 4. Use Case Diagram

Proposed Recommendation Engine / Machine Learning Process

The Recommendation Engine is the proposed intelligent component of TrainWise that will be integrated into the existing web-based employee training needs assessment system. At the current stage of development, TrainWise already functions as a training needs assessment and management platform

where users can register, log in, complete their profiles, submit Training Needs Assessment (TNA) forms, create or update Individual Development Plans (IDPs), receive notifications, and allow authorized personnel to monitor submissions, evaluations, and reports. However, the machine learning-based recommendation feature has not yet been fully developed or deployed in the current version of the system.

This section presents the planned machine learning process that will serve as the basis for the future implementation of the TrainWise Recommendation Engine. The proposed recommendation engine is designed to analyze submitted employee data and training-related inputs in order to generate more personalized and data-driven training recommendations. Once implemented, the engine will use XGBoost for structured data analysis and Natural Language Processing, specifically Sentence-BERT (SBERT), for analyzing textual inputs such as desired training, comments, and self-assessment responses.

The inclusion of this proposed component is necessary because the current system already collects important training needs assessment data that can later be used for intelligent analysis. These records include user profile information, department, role, employee type, training history, assessment responses, desired training programs, IDP records, and evaluation results. Instead of limiting the system to manual review and administrative monitoring, the proposed recommendation engine aims to support decision-making by transforming collected data into meaningful training suggestions.

Current System Status

TrainWise is now running as a web-based system for training needs assessment. It lets users register accounts, manage profiles, submit TNA forms, create IDPs, and handle evaluations, reports, notifications, and document exports. These features set the stage for adding recommendations in the future, since they help the system gather and store the data needed for machine learning.

At present, the system does not perform automated machine learning inference. Training recommendations are not yet created using a trained XGBoost model or SBERT-based semantic analysis. For now, the system is used to collect and manage data, which will help with building the machine learning part later. This way, the researchers can focus on getting the main features working before adding intelligent recommendations.

Proposed Machine Learning Component

The proposed machine learning component will be responsible for processing the data collected from the TNA form, user profile, training records, IDP submissions, and evaluation results. The component will be developed as a separate Python-based Recommendation API that will communicate with the existing TrainWise web application and MySQL database.

Once implemented, the process will begin when an end user submits a Training Needs Assessment form and desired training input. The submitted data

will be stored in the MySQL database and sent to the proposed Python Recommendation API for processing. The API will clean and prepare the data, analyze structured employee information using XGBoost, process textual input using SBERT, combine the results, and return recommended training programs to the main system.

This planned setup follows a modular design, where the main TrainWise web system handles user interaction, form submission, dashboard access, reporting, and database transactions, while the Python Recommendation API handles the machine learning and Natural Language Processing tasks. This separation will make the system easier to maintain and improve because the recommendation engine can be updated without fully changing the main web application.

Proposed XGBoost Component for Structured Data

XGBoost is planned to be used for analyzing structured employee data collected by TrainWise. Structured data refers to organized information that can be stored in database fields, such as department, employee type, designation, years of service, specialization, previous training, assessment responses, and competency-related information.

In the future implementation, this data will be converted into features that can be processed by the XGBoost model. The model will be trained to identify patterns that may indicate the employee's possible training needs or competency gaps. For example, employees from a specific department, role, or specialization

may require certain training programs based on their profile, assessment results, or past training history.

The purpose of using XGBoost is not to replace the judgment of HR administrators or evaluators. Instead, it will serve as a decision-support tool that can help identify possible training needs faster and more consistently. The generated recommendation may still be reviewed and validated by authorized personnel before being used as the final basis for employee development planning.

Proposed Natural Language Processing Component Using SBERT

In addition to structured data, TrainWise also collects textual inputs from users, especially through desired training fields, comments, suggestions, and open-ended assessment responses. These text-based inputs are important because users may describe their training needs in their own words. However, manual interpretation of these responses can be time-consuming, especially when many employees submit assessment forms.

To address this, the proposed system will use Sentence-BERT or SBERT for Natural Language Processing. SBERT will be used to convert user-submitted text and training program descriptions into sentence embeddings. These embeddings are numerical representations of sentence meaning. Through this process, the system can compare the meaning of the user's desired training input with the descriptions of available training programs in the database.

This will allow TrainWise to recommend training programs based on semantic similarity instead of exact keyword matching. For example, if a user enters “I want to improve my skills in data protection and online security,” the system may still match the input with training programs related to cybersecurity, information security, or data privacy even if the exact words are not identical. This makes the recommendation process more flexible and meaningful.

Proposed Recommendation Generation Process

The proposed recommendation generation process will begin when the user submits the Training Needs Assessment form and desired training information. The submitted data will first be stored in the MySQL database. After storage, selected data fields will be sent to the proposed Python Recommendation API.

The XGBoost component will analyze the structured employee data to identify possible training categories or competency gaps. At the same time, the SBERT component will analyze the user’s desired training text and compare it with the available training program descriptions. The system will then combine the structured data result and text similarity result to rank the most relevant training programs.

After ranking, the recommendation engine will return the recommended training programs to the TrainWise web application. These recommendations will be displayed on the user dashboard and saved in the database for future reference. The user may use the recommendations as a basis for creating or

updating the Individual Development Plan. HR administrators and evaluators may also monitor the results through the reporting and evaluation modules.

Proposed System Integration

The proposed Recommendation Engine will be integrated into TrainWise through a Python-based API. The existing TrainWise web application will continue to handle the user interface, role-based access, form submission, report generation, and database management. The Python API will handle the machine learning and NLP-related processes.

Once implemented, the PHP-based web application will send the necessary assessment and profile data to the Python API. The API will process the data using the planned XGBoost and SBERT components, generate the recommendation output, and send the result back to the main system. The recommendation results will then be stored in the MySQL database and displayed to the user.

This design allows the current system to remain functional while preparing for future intelligent features. It also allows the researchers to develop and test the recommendation engine separately before full deployment.

Algorithm Analysis

This section presents the algorithm analysis of the proposed intelligent recommendation engine of TrainWise. It covers the planned model evaluation approach through NLP Evaluation Metrics, and the Data Model Generation

process, which outlines the step-by-step procedure for building, training, testing, and validating the XGBoost and Sentence-BERT (SBERT) components of the system.

Model Evaluation

NLP Evaluation Metrics

The performance of the proposed NLP-based recommendation component is evaluated using five metrics once the model has been developed and sufficient labeled data are available.

Accuracy refers to the proportion of correctly predicted instances — both positive and negative — out of all test cases. It reflects the overall correctness of the recommendation output.

The formula to calculate the Accuracy is:

Accuracy

$$Accuracy = \frac{(TP + TN)}{(TP + FP + TN + FN)} \quad (1)$$

Precision measures the proportion of recommended training programs that are truly relevant to the employee's needs. A higher precision value indicates fewer irrelevant recommendations.

The formula to calculate the precision is:

Precision

$$Precision = \frac{TP}{(TP + FP)} \quad (2)$$

Recall measures the proportion of relevant training programs that the system successfully retrieves. A higher recall value ensures that important training recommendations are not missed.

The formula to calculate recall is:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score provides a balanced evaluation by combining precision and recall into a single metric. It is particularly useful when training program categories are unevenly distributed in the dataset.

The formula to calculate the F1 score is:

F1-Score

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) \frac{\text{Precision} + \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

Cosine Similarity evaluates the semantic closeness between the employee's desired training text and available training program descriptions. A value closer to 1.0 indicates a stronger match and is the primary evaluation metric for the SBERT component.

The formula to calculate the cosine similarity is:

Cosine Similarity

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \times \|B\|} \quad (5)$$

For the XGBoost component, which processes structured fields such as department, designation, specialization, teaching status, and years in LSPU, the metrics in formulas (1) through (4) are applied once labeled recommendation records become available. Expert validation may also be conducted by allowing HR administrators, college deans, or authorized evaluators to assess whether the recommended training programs align with the employee's profile and development needs.

Data Model Generation

The Data Model Generation outlines the planned step-by-step process for building the proposed recommendation model of TrainWise.

Model Selection

The proposed recommendation engine will use a hybrid approach combining XGBoost for structured data analysis and Sentence-BERT (SBERT) for Natural Language Processing. XGBoost was selected because it efficiently handles structured tabular data such as employee department, designation, specialization, years of service, and assessment records stored in the TrainWise MySQL database. SBERT was selected because it generates sentence

embeddings that capture semantic meaning, enabling the system to match employee-submitted desired training text with training program descriptions even when exact keywords do not match.

Data Collection and Preparation

The dataset will be sourced from the existing TrainWise MySQL database. Structured data will come from the users table, including fields such as department, designation, specialization, educational attainment, years in LSPU, teaching status, and position. Text-based data will come from the assessments table, specifically the desired skills, comments, and training history fields. Training program records maintained by the HR Administrator, including training titles, descriptions, categories, and target competencies, will serve as the reference dataset for recommendation generation. Before model training, the completeness and accuracy of these database records must be verified to ensure reliable output.

Data Annotation

Since the system does not yet have a fully labeled recommendation dataset, data annotation will be required before model training. Historical assessment records may be reviewed by HR personnel or subject matter experts to assign appropriate training program labels or categories based on each employee's profile and assessment responses. A small labeled dataset may be constructed with assistance from HR administrators and department heads to provide initial ground truth data for model evaluation.

Data Preprocessing

For structured data processed by XGBoost, preprocessing will include handling missing values, encoding categorical fields such as department, designation, and teaching status, and normalizing numerical fields such as years in LSPU. For text-based data processed by SBERT, preprocessing will include lowercasing all text, removing unnecessary characters and punctuation, tokenization, stop-word removal, and preparing inputs for SBERT embedding generation. Both employee desired training text and training program descriptions will undergo the same preprocessing steps to enable accurate semantic comparison.

Data Splitting

The available labeled dataset will be divided into 80% for training and 20% for testing. The training subset will be used to train the XGBoost model and prepare SBERT embeddings for training program descriptions, while the test subset will be used to evaluate the quality and accuracy of generated recommendations. This split follows standard machine learning practice and ensures that the model is evaluated on data it has not previously seen.

Model Experimentation

Before finalizing the recommendation engine, the researchers will experiment with different configurations of both models. For XGBoost, parameters such as the number of estimators, learning rate, and maximum tree

depth will be adjusted to identify the best-performing settings. For SBERT, different pre-trained model variants may be tested to determine which produces the most meaningful embeddings for training needs assessment text. Cosine similarity threshold values will also be explored to determine the appropriate cutoff score for filtering relevant training program matches.

Model Testing

Testing will be conducted using the reserved 20% test dataset. The XGBoost component will be evaluated by comparing predicted training categories against labeled ground truth records using accuracy, precision, recall, and F1-score. The SBERT component will be tested by evaluating cosine similarity scores between employee desired training text and training program descriptions. Selected outputs may also be reviewed by HR administrators or department heads to assess whether the recommendations are appropriate for the employee's profile and needs.

Model Development

The final development stage involves building the Python-based Recommendation API using FastAPI, which will serve as the bridge between the existing TrainWise PHP web application and the proposed machine learning components. The trained XGBoost model will be saved as a serialized model file. The SBERT model will generate embeddings for all training program descriptions stored in the database. When an employee submits a TNA form with desired training input, the API will preprocess the data, run the XGBoost prediction on

structured profile fields, compute SBERT cosine similarity scores for the textual input, combine the results, rank the training programs, and return the top recommendations to the TrainWise web application for display and storage.

Development Status and Limitation

At the current stage, the Recommendation Engine is considered a proposed intelligent feature of TrainWise. The existing system already supports the collection and management of training needs assessment data, but it does not yet include actual machine learning-based recommendation generation. The proposed use of XGBoost and SBERT will be implemented after sufficient data preparation, model development, API integration, and system testing.

Because of this, the current study must clearly distinguish between the existing system features and the proposed machine learning component. The existing system provides the operational foundation for training needs assessment, while the planned recommendation engine will serve as an enhancement that aims to improve the accuracy and personalization of training recommendations.

The recommendation engine will also be limited by the quality and quantity of data available in the system. Incomplete user profiles, unclear desired training inputs, limited training program records, or insufficient labeled data may affect the performance of the model. Furthermore, the proposed system will not include real-time model retraining or integration with external training platforms in its current planned scope.

Overall, the proposed Recommendation Engine is intended to strengthen TrainWise by adding intelligent analysis to the existing training needs assessment process. Once developed, XGBoost will be used to analyze structured employee data, while SBERT will be used to interpret textual training needs. This planned hybrid approach will allow the system to provide more relevant, personalized, and data-driven training recommendations while still allowing HR administrators and evaluators to review and validate the results.

Materials for Software Development

This section presents the software and hardware materials used in the development, deployment, and proposed enhancement of TrainWise: A Web and Mobile-Based Employee Training Needs Assessment and Recommendation System using XGBoost and Natural Language Processing. The materials were selected based on their relevance to the existing system modules, database operations, administrative processes, user interaction, document generation, and the planned integration of the machine learning-based recommendation engine.

Since the current TrainWise platform has already been developed as a PHP and MySQL-based web application, the materials are grouped into two categories. The first category includes the tools used for the existing TrainWise system, while the second category includes the proposed tools for the future development of the intelligent recommendation engine. This separation ensures that the paper does not misrepresent the machine learning component as already implemented.

Software. This section presents the software tools used in developing the existing TrainWise system and the software tools proposed for its enhancement, along with their specific roles in the development process.

Visual Studio Code serves as the primary source code editor for TrainWise, providing a unified environment for editing PHP, HTML, CSS, JavaScript, and SQL files across all system modules. It will also be used for writing and maintaining the Python-based Recommendation API during the proposed enhancement phase.

HTML defines the structure of every TrainWise web page, organizing the forms, tables, navigation menus, and dashboard layouts that users interact with. All role-specific pages — including the TNA submission form, IDP form, evaluation forms, and administrative monitoring pages — are built on HTML and rendered consistently across desktop and mobile browsers.

CSS and Tailwind CSS control the visual appearance and responsive layout of the TrainWise interface. Tailwind CSS provides utility-based design classes applied directly to interface components, ensuring consistent styling and adaptability across screen sizes without requiring a separate mobile application.

JavaScript handles client-side interactivity in TrainWise, including form validation, modal windows, dynamic content display, and real-time interface updates. It validates required fields before TNA and IDP submissions, reducing input errors and improving usability for all user roles.

PHP serves as the server-side scripting language of the existing TrainWise system, managing all major backend operations — including user authentication, session handling, role-based access, form processing, database communication, and report generation.

MySQL / MariaDB is the database management system of TrainWise, storing all user accounts, profile records, TNA submissions, IDP data, evaluation results, deadlines, notifications, and training program data. In the proposed enhancement, the database will be expanded with additional tables for recommendation outputs and similarity scores.

XAMPP and phpMyAdmin provide the local development and testing environment for the PHP and MySQL components of the system. phpMyAdmin serves as the graphical interface for managing database tables, records, and contents throughout development.

FPDF and jsPDF support the PDF document generation features of TrainWise, producing downloadable TNA reports, IDP documents, evaluation summaries, and other printable records required by users and administrators.

PHPMailer enables the automated email notification features of the system, sending registration updates, deadline alerts, and system reminders to users, evaluators, and administrators.

Web Browser (Google Chrome or Microsoft Edge) is the access point for all TrainWise users, since the platform is fully browser-based and requires no

installation. The system is designed to function consistently across updated browsers on any internet-connected device.

Web Hosting Environment. The existing TrainWise system is deployed on a PHP- and MySQL-compatible hosting environment (InfinityFree) that supports file deployment, database connections, user sessions, and web-based access for LSPU Los Baños Campus users. The proposed recommendation engine will require a separate Python-compatible hosting service for its API deployment.

Proposed Software for the Recommendation Engine

Python

Python is proposed as the main programming language for developing the intelligent recommendation engine. It is suitable for machine learning and Natural Language Processing because it supports libraries for data processing, model development, text analysis, and API deployment. In the proposed TrainWise enhancement, Python is intended to process structured data—such as department, role, employee type, specialization, designation, years of service, training history, and assessment records—and unstructured data such as desired training inputs, comments, suggestions, and training program descriptions, for the generation of recommendation outputs.

FastAPI

FastAPI is proposed as the framework for creating the Python-based Recommendation API. The API serves as the bridge between the existing PHP-based TrainWise system and the proposed machine learning component, allowing the PHP system to send selected user and assessment data to the machine learning service and receive recommended training programs in return. FastAPI supports RESTful API development, structured request handling, automatic documentation, and efficient data request processing. In the proposed architecture, the TrainWise PHP system continues to handle the main web application, while FastAPI handles the recommendation process as a separate, independently testable service.

XGBoost

XGBoost is proposed as the machine learning algorithm for analyzing structured employee and assessment data. It is intended to identify patterns from numerical and categorical data related to employee training needs, using input features such as department, role, employee type, educational attainment, specialization, designation, years of service, training history, and assessment-related records. In the proposed TrainWise recommendation engine, XGBoost supports prediction and ranking of training needs based on available structured data. Model performance cannot be claimed without actual training, testing, and evaluation; therefore, XGBoost is treated as a proposed future component until sufficient labeled data are available.

Sentence-BERT / SBERT

Sentence-BERT, or SBERT, is proposed as the Natural Language Processing model for analyzing textual training needs and training program descriptions. Unlike basic keyword matching, SBERT generates sentence embeddings that capture semantic meaning, enabling the system to compare the meaning of an employee's desired training input with the descriptions of available training programs even when exact keywords do not match. In the proposed TrainWise recommendation engine, SBERT processes text fields such as desired skills, training preferences, comments, suggestions, and training descriptions to generate more meaningful and context-aware recommendations.

Pandas, NumPy, and scikit-learn

Pandas, NumPy, and scikit-learn are proposed Python libraries for data preparation, preprocessing, and model evaluation. Pandas is used to organize and clean tabular data from the TrainWise database. NumPy supports numerical operations, particularly when working with vectors and arrays. scikit-learn handles feature encoding, dataset splitting, similarity computation, and evaluation metrics. These libraries support the preparation of user profile data, assessment records, and training program data before processing by XGBoost or SBERT, and may be used to evaluate recommendation quality through accuracy, precision, recall, F1-score, or similarity scores.

Python API Hosting

A separate Python-compatible hosting environment is proposed for the future deployment of the Recommendation API. Since the existing TrainWise

system runs on a PHP and MySQL-based environment, the machine learning component requires a different server setup that supports Python, model files, API routes, and necessary libraries. This approach keeps the existing TrainWise system stable while the recommendation engine is developed and deployed as an independent service.

Hardware

Table 2 shows the following hardware specifications are recommended for the development, testing, and future enhancement of TrainWise. The minimum specifications are sufficient for basic system access, form submission, database testing, and browser-based operation. However, the recommended specifications are more appropriate for development work, especially when running a local server, editing source code, managing the database, testing multiple browser sessions, and preparing the proposed machine learning component. For the future development of the recommendation engine, a device with at least 8 GB of RAM is recommended, while 16 GB is preferred for smoother data processing and model testing. If the researchers will process larger datasets or experiment with more advanced Natural Language Processing models, cloud-based computing or a higher-performance workstation may be considered.

Table 2. Hardware Specifications

Hardware	Minimum Specification	Recommended Specification
----------	-----------------------	---------------------------

Processor	Intel Core i3 or AMD equivalent	Intel Core i5 / Ryzen 5 or higher
RAM	4 GB	8 GB to 16 GB
Storage	160 GB available storage	256 GB SSD or higher
Operating System	Windows 10	Windows 10/11
Internet Connection	Required for online access and hosting	Stable broadband connection
Browser	Updated Chrome or Edge	Updated Chrome or Edge
Development Device	Laptop or desktop computer	Laptop or desktop computer with multitasking support

Project Development

For this study, the proponents employed the Agile Scrum Methodology as the software development life cycle model for the development and enhancement of TrainWise. Agile Scrum was selected because the system is composed of several modules that require continuous planning, development, testing, feedback, and improvement. This methodology allows the proponents to divide the project into manageable development activities called sprints, where each sprint produces a working system component or improvement.

Figure 5 shows the Agile Scrum methodology applied in the development and enhancement of TrainWise. The Agile Scrum process used in this study includes product backlog preparation, sprint planning, sprint backlog creation, sprint execution, daily scrum, sprint review, sprint retrospective, and system increment.

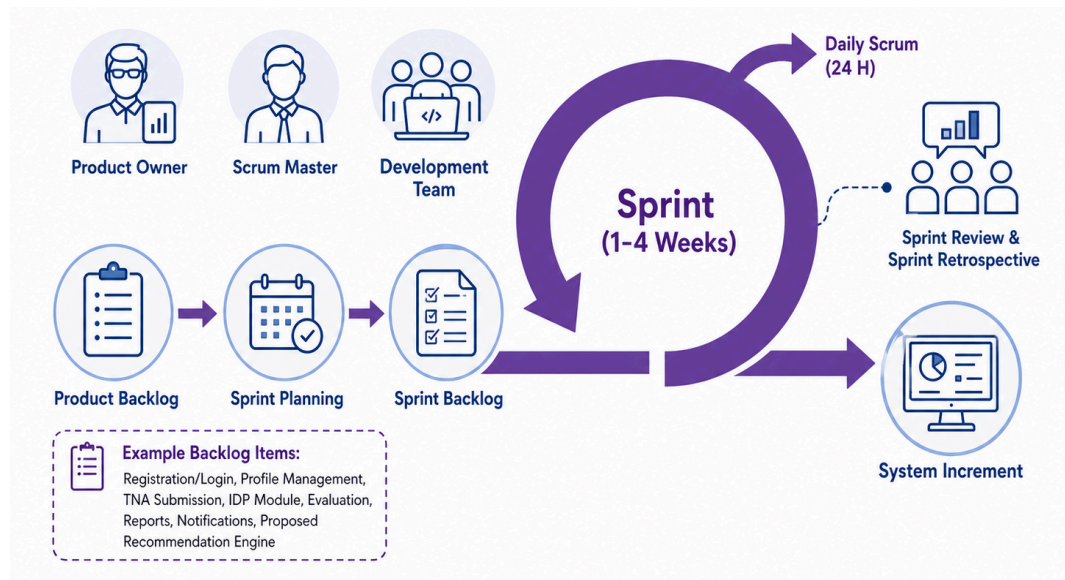


Figure 5. Agile Scrum Model

Product Backlog

The Product Backlog contains the list of features and requirements needed for the development and enhancement of TrainWise. The backlog was arranged according to priority, with the most important features listed first. Each item includes the estimated hours per day needed by the proponents to design, develop, test, and improve the feature.

Since TrainWise has different user roles, the Product Backlog was divided according to the main users of the system. This includes the backlog for regular users, HR Admin, Dean/Evaluator, and the proposed recommendation engine. This arrangement allows the proponents to identify the specific functions needed by each role and properly prioritize the development of each module.

Table 3 presents the Product Backlog for Users. This table includes the main functions needed by faculty members and non-teaching personnel when using TrainWise. These features focus on user access, profile completion, Training Needs Assessment submission, Individual Development Plan creation, notifications, document generation, and the viewing of proposed recommendation outputs once the recommendation engine is integrated.

Table 3. Product Backlog for Users

Prior ity	Features	Estimated hours per day
1	As a user, I want to register and log in to the system using my assigned role.	3 hrs
2	As a user, I want to complete and update my profile information.	4 hrs
3	As a user, I want to submit my Training Needs Assessment form.	5 hrs
4	As a user, I want to enter my training history, desired skills, and comments.	4 hrs
5	As a user, I want to create and submit my Individual Development Plan.	4 hrs
6	As a user, I want to receive notifications and reminders about deadlines and system updates.	3 hrs
7	As a user, I want to view or generate printable TNA and IDP documents.	4 hrs

8	As a user, I want to view recommended training programs once the recommendation engine is integrated. (<i>Proposed</i>)	4 hrs
---	---	-------

Table 4 presents the Product Backlog for HR Admin. This table includes the administrative features needed to manage users, monitor submissions, set deadlines, generate reports, and maintain training-related records. The HR Admin serves as the main system controller; therefore, the features under this backlog focus on management, monitoring, documentation, and system administration.

Table 4. Product Backlog for HR Admin

Priori ty	Features	Estimated hours per day
1	As an HR Admin, I want to approve or decline user registrations.	3 hrs
2	As an HR Admin, I want to manage user accounts and role-based access.	4 hrs
3	As an HR Admin, I want to set and manage TNA submission deadlines.	3 hrs
4	As an HR Admin, I want to monitor TNA submissions and IDP records.	4 hrs
5	As an HR Admin, I want to view employee training records and evaluation results.	4 hrs

6	As an HR Admin, I want to generate reports and export PDF documents.	4 hrs
7	As an HR Admin, I want to manage notifications and system announcements.	3 hrs
8	As an HR Admin, I want to manage training program records for future recommendation matching. <i>(Proposed)</i>	4 hrs

Table 5 presents the Product Backlog for the Dean/Evaluator. This table focuses on the evaluation-related functions of TrainWise. Since the Dean or evaluator is responsible for reviewing employee submissions by department, the backlog includes access to department records, TNA review, evaluation, filtering, and viewing of evaluation summaries.

Table 5. Product Backlog for Dean/Evaluator

Priori ty	Features	Estimated hours per day
1	As a Dean/Evaluator, I want to log in securely using my assigned account.	2 hrs
2	As a Dean/Evaluator, I want to view employee submissions by department.	4 hrs
3	As a Dean/Evaluator, I want to review Training Needs Assessment records.	4 hrs
4	As a Dean/Evaluator, I want to evaluate employee submissions.	5 hrs
5	As a Dean/Evaluator, I want to filter records by department, status, and employee type.	4 hrs

6	As a Dean/Evaluator, I want to view evaluation summaries for monitoring and documentation.	4 hrs
---	--	-------

Sprint Planning

As stated in the Sprint Backlog, selected items from the Product Backlog were moved into specific development tasks that the proponents planned to accomplish within each sprint. These tasks served as a guide in designing, coding, testing, and improving each module of TrainWise. The Sprint Backlog includes the feature to be developed, the specific task to be performed, and the estimated hours per day needed to complete each task.

Table 6 presents the Sprint Backlog for Users, which details the specific features and tasks selected for development during the sprint. These items served as the proponents' guide in designing, coding, testing, and improving the user-side modules of TrainWise. The sprint backlog includes tasks for user registration, role-based access, profile management, Training Needs Assessment submission, Individual Development Plan creation, notifications, printable documents, and the proposed interface for viewing future recommendation results.

Table 6. Sprint Backlog for Users

Prior	Features	Task	Estimated hours
-------	----------	------	-----------------

ity			per day
1	User registration and login	Design and develop user registration, login validation, and session handling.	3 hrs
2	Role-based access	Create access control for Faculty and Non-Teaching Personnel users.	3 hrs
3	Profile management	Develop profile completion and update forms for employee information.	4 hrs
4	TNA submission	Design and develop the Training Needs Assessment form with validation.	5 hrs
5	Training history and desired skills	Store training history, desired skills, and comments in the database.	4 hrs
6	IDP creation	Design and develop the Individual Development Plan creation and submission module.	4 hrs
7	Notifications	Implement user notifications and reminders for deadlines and updates.	3 hrs
8	Printable documents	Develop PDF export for TNA and IDP documents.	4 hrs
9	Recommended training programs	Prepare the user interface for displaying future recommendation results. <i>(Proposed)</i>	4 hrs

Table 7 shows the sprint backlog for the HR Admin side.

Table 7. Sprint Backlog for HR Admin

Priority	Features	Task	Estimated hours per day
1	User approval	Develop approval and decline functions for newly registered users.	3 hrs
2	User management	Create user account management and role assignment functions.	4 hrs
3	Deadline management	Develop deadline setting and monitoring for TNA submissions.	3 hrs
4	Submission monitoring	Create monitoring functions for TNA submissions and IDP records.	4 hrs
5	Admin dashboard	Develop dashboard cards, summary counts, and administrative links.	4 hrs
6	Reports	Create report filtering, viewing, and monitoring functions.	4 hrs
7	PDF export	Develop PDF export for TNA, IDP, evaluation, and summary reports.	4 hrs
8	Notifications	Implement notification and announcement management for users.	3 hrs

9	Training program records	Prepare database structure for future training program records. <i>(Proposed)</i>	4 hrs
10	Recommendation result storage	Prepare proposed storage for recommendation results and similarity scores. <i>(Proposed)</i>	5 hrs

Table 8 shows the sprint backlog for the Dean/Evaluator side.

Table 8. Sprint Backlog for Dean/Evaluator

Priority	Features	Task	Estimated hours per day
1	Dean/Evaluator login	Design and develop secure login access for Dean/Evaluator accounts.	2 hrs
2	Department records	Develop a page for viewing employee submissions by department.	4 hrs
3	TNA review	Create a detailed view for reviewing Training Needs Assessment records.	4 hrs
4	Evaluation form	Develop department-based evaluation forms and rating	5 hrs

		submission.	
5	Filtering	Add filtering by department, employee type, and submission status.	4 hrs
6	Evaluation summary	Create evaluation summary views for monitoring and documentation.	4 hrs

Table 9 shows the sprint backlog for the proposed recommendation engine preparation.

Table 9. Sprint Backlog for Proposed Recommendation Engine Preparation

Priori ty	Features	Task	Estimated hours per day
1	Data preparation	Identify employee profile, TNA, IDP, and evaluation fields needed for recommendation processing. <i>(Proposed)</i>	5 hrs
2	Training program dataset	Prepare the proposed structure for training program names, categories, descriptions, and target competencies. <i>(Proposed)</i>	4 hrs
3	Python Recommendation API	Design the proposed API flow between the PHP-based TrainWise system and the Python recommendation service.	5 hrs

		(Proposed)	
4	XGBoost processing	Prepare structured data features such as department, role, designation, years of service, and training history. (Proposed)	6 hrs
5	SBERT text matching	Prepare the process for comparing desired skills and comments with training program descriptions using semantic similarity. (Proposed)	6 hrs
6	Recommendation output	Prepare the planned interface and storage for recommendation results, similarity scores, and model outputs. (Proposed)	5 hrs

Daily Scrum

The Daily Scrum was conducted to monitor the progress of the development activities. During this short meeting, the proponents discussed completed tasks, current development concerns, encountered errors, and the next tasks to be performed. This allowed the team to identify issues early and make necessary adjustments during the sprint.

For TrainWise, the Daily Scrum focused on checking the progress of user authentication, profile management, TNA submission, IDP creation, evaluation pages, report generation, database functions, interface design, and system

testing. For the proposed recommendation engine, Daily Scrum discussions may include data preparation, database enhancement, Python API planning, XGBoost feature requirements, SBERT text matching, and recommendation result validation.

Sprint Execution

Sprint Execution refers to the actual development of the tasks identified in the Sprint Backlog. During this phase, the proponents coded, tested, revised, and improved the selected system features. The existing TrainWise system was developed using PHP, MySQL/MariaDB, HTML, CSS, Tailwind CSS, JavaScript, and supporting tools for PDF generation and email notification.

For the proposed recommendation engine, Sprint Execution will involve the preparation of the Python-based Recommendation API, structured data processing using XGBoost, and textual similarity matching using Sentence-BERT. However, since the machine learning component is not yet fully implemented, this part remains a planned enhancement that will be developed after sufficient data preparation, model development, and validation.

Sprint Review

The Sprint Review was conducted at the end of each sprint to evaluate the completed features and determine whether they met the expected requirements. During this phase, the proponents reviewed the functionality of the system

modules and identified possible improvements based on adviser, evaluator, and user feedback.

For the existing TrainWise system, Sprint Review involved checking the registration and login process, profile completion, TNA submission, IDP creation, evaluation module, notification feature, report generation, and PDF export. For the proposed recommendation engine, the review will focus on checking whether the planned data structure, API flow, model input requirements, and recommendation output design are appropriate for future integration.

Sprint Retrospective

The Sprint Retrospective was conducted after each Sprint Review to identify what went well, what problems occurred, and what improvements should be applied in the next sprint. This phase helped the proponents improve their development process, task distribution, coding practices, testing activities, and documentation.

In TrainWise, the Sprint Retrospective helped the proponents identify improvements related to user interface consistency, database structure, form validation, role-based access, report formatting, and usability. It also helped clarify that the recommendation engine should be presented as a proposed enhancement rather than a completed machine learning component.

System Increment and Deployment

At the end of each sprint, a working system increment was produced. Each increment represented a completed or improved part of TrainWise. These increments were tested and reviewed before being integrated into the main system.

The existing TrainWise system was deployed as a PHP and MySQL-based web application for use by LSPU Los Baños Campus. The proposed recommendation engine will be developed as a separate Python-based component in a future enhancement phase. This setup allows the current system to remain operational while the machine learning process is prepared, tested, and integrated through an API.

Through Agile Scrum Methodology, the proponents were able to organize the development of TrainWise into manageable and measurable tasks. The use of Product Backlog, Sprint Planning, Sprint Backlog, Daily Scrum, Sprint Review, and Sprint Retrospective supported the systematic development of the existing system and the structured preparation of the proposed recommendation engine. This methodology is suitable for the project because TrainWise requires continuous improvement, user feedback, module-based testing, and future integration of machine learning and Natural Language Processing features.

Project Testing and Evaluation

This section presents the testing and evaluation procedures conducted in fulfillment of the three specific objectives of the study. In accordance with the objectives, the testing procedures are organized to: (1) validate the design of the proposed XGBoost and Natural Language Processing-based recommendation model through a planned testing framework; (2) verify the correctness and readiness of the enhanced TrainWise application — covering user registration, TNA form submission, IDP creation, evaluation management, report generation, and notification delivery — through functional and non-functional software testing; and (3) formally evaluate the existing and enhanced system using the Technology Acceptance Model (TAM) and the ISO/IEC 25010 Software Quality Model in terms of usability, functionality, performance efficiency, and user acceptance. The testing procedures are applied to both the existing fully developed system modules and the planned recommendation engine, which is currently under preparation for integration in a subsequent development phase.

Furthermore, the evaluation process employs the ISO/IEC 25010 Software Quality Model and the Technology Acceptance Model (TAM) as evaluation frameworks. The ISO 25010 model assesses the system's product quality characteristics, while TAM measures perceived usefulness and ease of use among different user groups. The results of these evaluations serve as the basis for improving the system and determining its readiness for institutional deployment at Laguna State Polytechnic University (LSPU) Los Baños Campus.

Software Testing

Software testing ensures that each module and component of TrainWise is properly validated before deployment. The proponents are to conduct multiple levels of testing — functional and non-functional — to confirm that the system performs as intended for all user roles: employees (faculty and staff), Unit Heads, College Deans, and the Admin.

Figure 8 shows the project testing and evaluation of the system. The first phase of this system's development is unit testing, which verifies that the system is operational. This is followed by integration testing, which focuses on the system architecture's modules and includes four additional submodules: compatibility, performance, stress, and load testing. Then there is system testing, which verifies that the system is operating as intended, and finally acceptance testing, which tests the system after it has been implemented.



Figure 6. Levels of Software Testing

Unit Testing

Unit testing is the first level of software testing to be conducted during the development of TrainWise. In this phase, each individual module and component of the system is tested in isolation to verify that it functions correctly according to its intended design. The proponents test the core system modules—including the

User Registration and Login Module, TNA Form Module, IDP Module, Evaluation Module, Notification Module, and Report Generation Module—independently before integrating them into the full system. Each module is subjected to test cases covering both valid and invalid inputs, boundary values, and expected system responses.

Test data consists of both simulated and actual records drawn from the TrainWise database, including valid and invalid credentials, sample TNA form entries, IDP records, and evaluation records. All test data are selected to represent both standard use cases and edge-case conditions such as missing fields, late submissions, and unauthorized access attempts.

Integration Testing

Integration testing is to be conducted after unit testing to verify that the individual modules of TrainWise work correctly when combined. This phase focuses on testing the interaction between modules, ensuring that data flows accurately from one component to another and that the integrated system produces the expected outputs.

The proponents follow the system's operational flow as the basis for integration test cases, beginning with user registration through to final report generation. For example, integration testing verifies that when a user submits a Training Needs Assessment form, the data is correctly captured, stored in the assessments table, and made visible to the assigned evaluator on their dashboard. Integration between the IDP module and the approval workflow is

also tested to confirm that data flows correctly across all related database tables. Test data traces actual records across multiple linked database tables to verify referential integrity and status transitions throughout the approval workflow.

System Testing

System Testing is to be conducted after all individual modules have passed unit and integration testing. In this phase, the proponents assess the complete TrainWise system by simulating real-world usage scenarios to evaluate the system's end-to-end functionality, data integrity, and operational readiness. The objective is to verify that TrainWise performs as an integrated whole and meets all defined functional and non-functional requirements.

The proponents simulate complete workflow scenarios for each user role — employee, Unit Head, Dean/Evaluator, and Admin — and verify that the system handles edge cases, incorrect inputs, session management, and data consistency correctly. Error handling mechanisms are also tested to confirm that the system responds appropriately to unexpected inputs or network interruptions. Initial feedback gathered from test users during this phase is incorporated to further refine the system interface and functionality prior to formal user acceptance testing.

Acceptance Testing

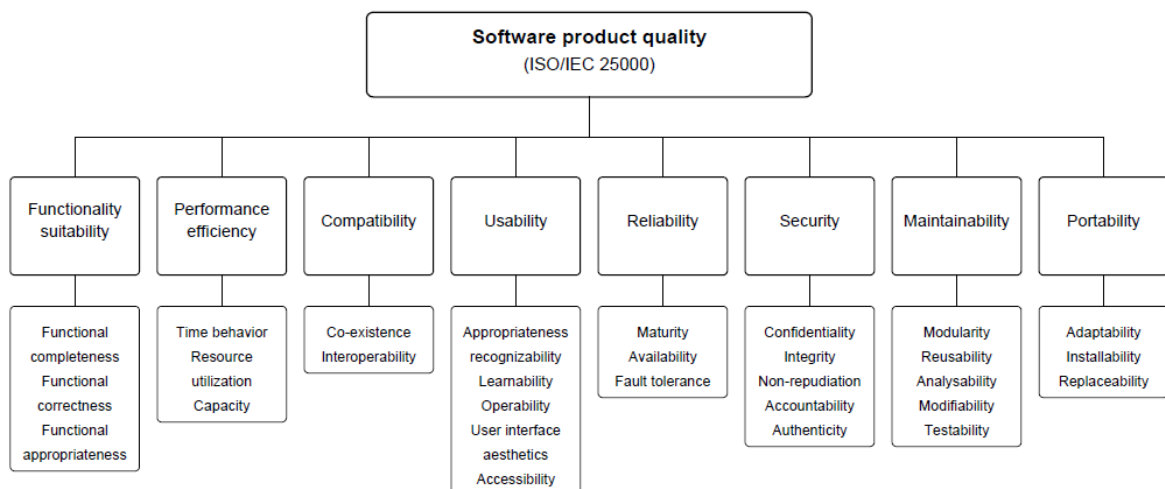
Acceptance Testing is the final phase of software testing and is to be conducted to formally confirm that the system satisfies the agreed functional

requirements and user expectations of its primary stakeholders: employees (faculty and staff), Unit Heads, College Deans, and the system administrator.

During acceptance testing, the proponents present the fully developed TrainWise system to the respondents under realistic operational conditions. Each user group is guided through the specific system functions relevant to their role, and respondents are asked to confirm whether each function performed as expected and whether the system addressed their actual work needs.

Evaluation Procedure

Standards such as ISO 25010, which outline the key aspects of product quality, are in place to ensure the quality of the system to be developed. The quality model regarded as the cornerstone of a system for evaluating product quality is described in the ISO 25010 standard. Eight quality characteristics that describe a software product's features for evaluation make up the ISO 25010 product quality model, as shown in Figure 9.



The respondents — employees (faculty and staff), Unit Heads, College Deans, and IT experts — evaluate the developed system using the ISO 25010 software standard to assess the performance and general features of TrainWise. The ISO 25010 model is the most widely used standard for determining the consistency and quality of any software program. Assessments include both functional and non-functional aspects, as well as internal and external design and structure.

For evaluating the system, the proponents conduct beta testing with employees, Unit Heads, College Deans, and IT experts. The ISO 25010 quality model serves as the standard for the Likert Scale. The Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, and Portability are the basis for testing and evaluating TrainWise. The Likert Scale is answerable by five responses: Strongly Agree (5), Agree (4), Neutral (3), Disagree (2), and Strongly Disagree(1).

Functional Suitability. One of the most important parts is software that must suit the goals and needs of the user. The proponents demonstrate that TrainWise correctly fulfills all user requirements — including TNA form submission, IDP creation, evaluation review, report generation, and notification delivery — to achieve acceptance of the solution as effective and efficient.

Performance Efficiency. Software execution is an important factor in selecting a product to use. The proponents assess whether TrainWise responds quickly to user interactions, processes form submissions without delay, and

generates PDF reports and email notifications within acceptable time limits, representing the overall capability of the system.

Compatibility. The proponents ensure client satisfaction by verifying that TrainWise functions consistently across a variety of browsers (Google Chrome, Mozilla Firefox, Microsoft Edge), operating systems (Windows, macOS, Android, iOS), and device types (desktops, laptops, and smartphones).

Usability. The product's worth and utility are readily apparent to users. The proponents design TrainWise to be accessible and easy to navigate for all user roles — employees, Unit Heads, College Deans, and administrators — including users who may vary in their level of digital literacy.

Reliability. As part of evaluating quality through analysis and determining accuracy, IT experts examine the application's reliability in consistently saving data correctly, maintaining session integrity across concurrent users, and recovering gracefully from unexpected interruptions.

Security. The IT professional examines the application's data protection measures, including session management, role-based access control (RBAC), input validation and sanitization implemented through the `sanitize_input()` function, and compliance with data privacy regulations.

Maintainability. The IT professional examines the application's capacity to determine whether it can be efficiently updated, extended, or restructured over

time — including its readiness for the planned integration of the XGBoost-NLP recommendation engine.

Portability. The proponents ensure that the system's functionality can be transferred from one environment to another — such as from the current InfinityFree hosting environment to alternative servers — with minimal modification to the PHP-MariaDB architecture.

Furthermore, initial testing is to be conducted by the developers during the development of TrainWise to discover and correct any flaws. The developers test every module and functionality using the created planned procedure to check and examine accuracy, reliability, efficiency, effectiveness, user-friendliness, and complete functionality — to determine whether the system achieves the desired features and requirements specified by employees and institutional stakeholders.

. The proponents ensure that the system's functionality can be transferred from one environment to another — such as from the current InfinityFree hosting environment to alternative servers — with minimal modification to the PHP-MariaDB architecture.

Technology Acceptance Model (TAM)

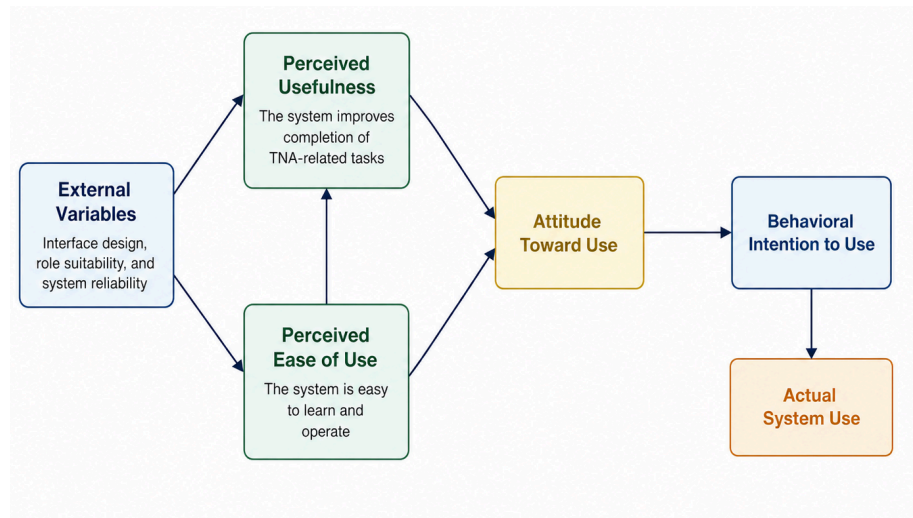


Figure 8. Quality Characteristics of the Technology Acceptance Model

The Technology Acceptance Model (TAM) is an information systems theory that explains how users come to accept and use a new technology. TAM highlights that actual system utilization — the point at which individuals actively use a technology — is influenced by several key factors. Traditionally, TAM identifies two primary factors that impact acceptance: (1) perceived usefulness, which reflects how beneficial the user believes the technology will be in enhancing their work performance; and (2) perceived ease of use, which represents how effortless they find the technology to operate. Additional factors also contribute to technology acceptance, including attitude toward use, which reflects the user's overall disposition toward adopting the system, and behavioral intention to use, which captures the user's plan to continue using the technology in their regular institutional work.

The model takes into account the user's readiness to integrate the technology into their daily tasks. Together, these factors shape the behavioral intention to use the technology, ultimately leading to actual system usage. This evaluation framework provides a structured way to identify and address barriers to adoption, ensuring that TrainWise meets the expectations and requirements of its users at LSPU Los Baños Campus. The TAM model also emphasizes iterative feedback, enabling the proponents to refine the system based on user perceptions and experiences. By understanding these dynamics, the proponents can prioritize features and functions that align with user needs, increasing the likelihood of successful institutional implementation and long-term utilization.

The proponents use the TAM framework to determine the extent to which TrainWise is easy to use and how each module is functional and reliable — and whether the system is genuinely useful to employees, Unit Heads, and administrators. This research collects data and responses from the conducted survey. As a result, it is important to evaluate the effectiveness of the system as well as how user requirements such as interface design and system benefits are addressed.

Table 10. Four-Point Likert Scale for TAM Evaluation

Degree	Range	Description
4	3.26 – 4.00	Strongly Acceptable

3	2.51 – 3.25	Acceptable
2	1.76 – 2.50	Not Acceptable
1	1.00 – 1.75	Strongly Not Acceptable

For the planned XGBoost-NLP recommendation engine, an additional TAM dimension will assess users' perceived usefulness and perceived ease of use of the automated training recommendation feature once it has been fully developed and integrated into TrainWise.

Planned Evaluation of the Recommendation Engine

The XGBoost and Natural Language Processing-based training recommendation engine is a planned enhancement of TrainWise that is not yet implemented at the time of this writing. The proponents recognize that the recommendation engine — the central machine learning component of the proposed system — requires several prerequisites before it can be properly developed, integrated, and evaluated. These prerequisites include the accumulation of a sufficient volume of labeled TNA data from actual system users, feature engineering based on structured TNA profiles (training history, desired skills, years in LSPU, position, department, and teaching status), model training and validation using XGBoost for classification and Sentence-BERT (SBERT) for textual similarity matching, and the development of a Python-based

RESTful API (using Flask or FastAPI) for serving real-time recommendations to the TrainWise front-end.

Once the recommendation engine is developed, its evaluation will involve the following planned approaches: First, the recommendation relevance will be assessed by computing metrics such as precision, recall, and F1-score by comparing the system's recommended training programs against expert-validated ground truth labels. Second, a user evaluation survey will be administered to employees and Unit Heads to assess the perceived usefulness and accuracy of the recommendations through the TAM framework. Third, an additional ISO 25010 evaluation round will be conducted specifically targeting the AI-driven recommendation module, focusing on Functional Suitability (correctness of recommendations), Performance Efficiency (response time of the recommendation API), and Reliability (consistency of outputs across multiple inference calls).

Table 11 outlines the planned evaluation metrics and approaches for the XGBoost-NLP recommendation engine, serving as a development roadmap for the next phase of TrainWise enhancement.

Table 11. Planned Evaluation Framework for the Recommendation Engine

Evaluation Aspect	Component Evaluated	Metric	Evaluation Approach	Target
Model Accuracy	XGBoost Classification	Accuracy, Precision, Recall, F1-Score,	Computed on held-out test set from TNA dataset	$\geq 80\%$ accuracy target

Similarity Score Analysis				
Recommendation Relevance	SBERT Similarity Matching	Cosine Similarity Score	Compared against expert-validated training program labels	High similarity scores (≥ 0.75) for top-ranked recommendations
API Response Time	Flask/FastAPI Inference Endpoint	Response Time (seconds)	Measured under normal load conditions (single and concurrent requests)	< 5 seconds per request
Perceived Usefulness (TAM)	End-User Survey	TAM Mean Score	4-point Likert Scale survey administered to employees and Unit Heads	Mean score ≥ 3.26 (Strongly Agree)
Perceived Ease of Use (TAM)	End-User Survey	TAM Mean Score	4-point Likert Scale survey on recommendation display and interface	Mean score ≥ 3.26 (Strongly Agree)
Functional Suitability (ISO 25010)	Beta Testing with Respondents	ISO 25010 Mean Score	5-point Likert Scale evaluation by IT experts and evaluators	Mean score ≥ 3.81 (Good)
System Reliability	IT Expert Review	ISO 25010 Mean Score	Evaluation of consistency and stability of recommendation outputs	Mean score ≥ 3.81 (Good)

The proponents emphasize that Table 18 represents a planned evaluation framework that will be executed once the recommendation engine has been fully developed and integrated into TrainWise. This roadmap ensures that the machine learning component will be evaluated rigorously — both from a technical performance standpoint and from the perspective of actual user acceptance — before it is released as a production feature of the system.

Statistical Treatment of Data

The proponents will apply the following statistical methods to analyze the data collected from the evaluation instruments:

Frequency and Percentage. The frequency of each response option is counted and expressed as a percentage of the total number of respondents using the formula: $\% = (f / N) \times 100$, where f is the frequency of a given response and N is the total number of respondents. This allows the proponents to identify the distribution of responses across evaluation items.

Weighted Mean. The weighted mean is computed for each evaluation item to produce an overall score that reflects the average rating assigned by respondents. For the TAM 4-point scale, the formula is: $\bar{x} = [(SA \times 4) + (A \times 3) + (D \times 2) + (SD \times 1)] / N$. For the ISO 25010 5-point scale, the formula is: $\bar{x} = [(SA \times 5) + (A \times 4) + (N \times 3) + (D \times 2) + (SD \times 1)] / N$, where SA = Strongly Agree, A = Agree, N = Neutral, D = Disagree, and SD = Strongly Disagree. The resulting mean is interpreted using the descriptive ranges presented in Tables 13 and 14.

Cronbach's Alpha. To measure the internal consistency and reliability of the evaluation instruments, the proponents compute Cronbach's alpha for each

TAM dimension. A Cronbach's alpha value of 0.70 or above is considered acceptable for research instruments. This statistical measure ensures that the survey items within each TAM dimension consistently measure the same underlying construct, supporting the credibility of the overall evaluation results.

Stratified Analysis. The proponents analyze evaluation results by respondent groups — employees (faculty and staff), Unit Heads/Deans, and IT experts — to identify differences in perceptions across user roles. This stratified analysis provides a more nuanced understanding of how different stakeholders experience TrainWise and highlights areas where targeted improvements may be needed for specific user groups.

Population of the Study

The population of this study consists of the primary users and technical evaluators of the TrainWise system at Laguna State Polytechnic University (LSPU) Los Baños Campus. The respondents are categorized into three groups that represent the complete range of system interaction: employees (faculty and staff) as primary end-users of the TNA Form, IDP, and Notification modules; Unit Heads and College Deans as institutional evaluators who use the assessment review and approval workflow; and IT experts who evaluate the system's technical quality from a professional standpoint, including its architecture and the planned recommendation component. These groups are selected because their combined perspectives — operational, administrative, and technical — ensure a comprehensive evaluation that is aligned with all three objectives of the study.

Table 12 presents the distribution of respondents involved in the testing and evaluation of TrainWise.

Table 12. Respondents of the Research Project

Category of Respondents	Sample Size
Employees (Faculty and Staff)	30
Unit Heads and College Deans/Evaluators	10
IT Experts/Professionals	3
Total	43

Sampling Design

The proponents employ purposive sampling as the primary sampling strategy for this study. This non-probability sampling technique is appropriate because the evaluation of TrainWise requires respondents who have direct, role-based experience with the system or possess the technical expertise necessary to assess its quality characteristics meaningfully. Respondents are selected from the LSPU Los Baños Campus community based on their designated role within the system — as employees, evaluators, or IT professionals — rather than through random selection.

In addition, stratified sampling is applied to organize the selected respondents into three distinct subgroups corresponding to their user role: employees (faculty and staff), Unit Heads and College Deans, and IT experts.

This stratification ensures that evaluation data is gathered from each relevant stakeholder group and that the analysis reflects role-specific perceptions of the system's usability, functionality, and technical quality. The combination of purposive and stratified sampling ensures that the study gathers targeted, representative, and contextually meaningful evaluation data from across the full spectrum of TrainWise users.

Data Collection Instrument

The proponents use structured questionnaire surveys as the primary data collection instruments, distributed to all respondents following the acceptance testing phase. Two sets of questionnaires are prepared and administered.

The first set is based on the ISO/IEC 25010 Software Quality Model and assesses the system's product quality across eight characteristics: Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, and Portability. This questionnaire uses a 5-point Likert Scale with response options of Strongly Agree (5), Agree (4), Neutral (3), Disagree (2), and Strongly Disagree (1). Each respondent group evaluates the quality dimensions applicable to their role: IT experts assess all eight characteristics with emphasis on Security, Maintainability, and Portability; Unit Heads and Deans focus on Functional Suitability, Usability, and Reliability; and employees assess Functional Suitability, Usability, and Performance Efficiency based on their direct system experience.

The second set is based on the Technology Acceptance Model (TAM) and measures user acceptance across four dimensions: Perceived Usefulness, Perceived Ease of Use, Attitude Toward Use, and Behavioral Intention to Use. This questionnaire uses a 4-point Likert scale with response options of Strongly Acceptable (4), Acceptable (3), Not Acceptable (2), and Strongly Not Acceptable (1). The 4-point scale is intentionally designed without a neutral midpoint, encouraging respondents to take a definitive position regarding their acceptance of the system. For the planned XGBoost-NLP recommendation engine, an additional set of TAM-based items will be prepared and administered separately once the module has been fully developed and integrated.

Both questionnaires are made available in physical printed copies and as Google Form digital surveys to ensure accessibility for all respondents. Interview sessions are also conducted as supplementary instruments to gather qualitative insights on the system's usability, institutional relevance, and potential improvements. All instruments are reviewed for clarity, relevance, and appropriateness prior to distribution, and are aligned with the specific features and workflow of TrainWise

Statistical Analysis of Data

The data gathered from the evaluation instruments are analyzed using the following statistical methods.

Frequency and Percentage Distribution. The frequency of each response option is tallied and converted to a percentage of the total number of respondents using the formula:

$$\% = (f / N) \times 100$$

Where f is the frequency of a given response and N is the total number of respondents. This allows the proponents to identify the distribution of responses across each evaluation item and determine which system characteristics are rated most and least favorably by respondents.

Weighted Mean. The weighted mean is computed for each evaluation item to derive a representative average score reflecting the collective rating of all respondents. For the ISO 25010 5-point scale, the formula is:

$$\bar{x} = [(SA \times 5) + (A \times 4) + (N \times 3) + (D \times 2) + (SD \times 1)] / N$$

Where: $\bar{x}$ = Mean; N = Total number of respondents; SA = Strongly Agree; A = Agree; N = Neutral; D = Disagree; SD = Strongly Disagree.

For the TAM 4-point scale, the formula is:

$$\bar{x} = [(SA \times 4) + (A \times 3) + (D \times 2) + (SD \times 1)] / N$$

Where: $\bar{x}$ = Mean; N = Total number of respondents; SA = Strongly Acceptable; A = Acceptable; NA = Not Acceptable; SNA = Strongly Not Acceptable.

The mean scores are interpreted using the descriptive ranges in Tables 13 and 14.

Table 13. Evaluation Degree, Range, and Description for ISO 25010

Degree	Range	Description
5	4.21 – 5.00	Strongly Agree
4	3.81 – 4.20	Agree
3	2.61 – 3.80	Neutral
2	1.81 – 2.60	Disagree
1	1.00 – 1.80	Strongly Disagree

Table 14. Evaluation Degree, Range, and Description for TAM

Degree	Range	Description
4	3.26 – 4.00	Strongly Acceptable
3	2.51 – 3.25	Acceptable
2	1.76 – 2.50	Not Acceptable
1	1.00 – 1.75	Strongly Not Acceptable

Cronbach's Alpha. The internal consistency of the TAM evaluation instrument is measured using Cronbach's alpha for each TAM dimension. A coefficient of 0.70 or above is considered acceptable, confirming that the survey

items within each dimension reliably measure the same underlying construct and supporting the credibility of the TAM analysis.

Stratified Group Analysis. Evaluation results are further analyzed by respondent category — employees, Unit Heads and Deans, and IT experts — to examine differences in perception across user roles. This stratified analysis provides a nuanced understanding of how different stakeholders experience TrainWise and identifies areas where targeted system refinements may be needed for specific groups. All data are encoded and analyzed using Microsoft Excel.

Ethical Consideration

In the development, testing, and evaluation of TrainWise, the proponents are committed to upholding the highest standards of ethical research practice. Given that the system processes sensitive institutional data — including employee training histories, competency assessments, individual development plans, and departmental evaluation records — the protection of respondents' privacy, dignity, and data rights is treated as a fundamental obligation throughout the study.

Informed Consent. Prior to any data collection activity, all respondents are provided with a clear explanation of the study's purpose, the scope of their participation, and how their data will be used. Informed consent forms written in plain, accessible language are distributed and obtained from each respondent before they participate in testing or complete any evaluation questionnaire.

Respondents are explicitly informed that participation is entirely voluntary and that they may withdraw at any time without consequence.

Confidentiality and Data Privacy. All personal information and evaluation data collected from respondents are treated with strict confidentiality. Individual responses are not disclosed to any unauthorized party and are used exclusively for evaluating and improving the TrainWise system. The study is conducted in full compliance with Republic Act No. 10173, or the Data Privacy Act of 2012, which governs the collection, processing, and protection of personal data in the Philippines. The role-based access control, session management, and input sanitization mechanisms built into TrainWise further ensure that employee records stored in the system are accessible only to authorized users.

Anonymity. Where applicable, individual responses are anonymized during data analysis so that no specific respondent can be identified from the reported results. Aggregate mean scores and percentage distributions are used in presenting evaluation outcomes, and no personal identifiers are included in the final research document.

Fairness and Non-Discrimination. All respondents are treated equitably throughout the evaluation process. Evaluation instruments are designed to be

free of leading or biased questions, and all respondent groups are given equal opportunity to assess the system without coercion or undue influence.

Integrity of the Research Process. The proponents maintain transparency and academic integrity in all phases of the study. The planned XGBoost-NLP recommendation engine is consistently identified as a proposed enhancement not yet implemented, and all related evaluation content is clearly framed as a planned framework rather than an actual result, to avoid any misrepresentation to the panel, the institution, or future readers of this study.

All collected evaluation forms and digital survey responses are stored securely and are accessible only to the research team. Digital data are retained only for the duration necessary to complete the study and are disposed of properly thereafter, in accordance with ethical and institutional data handling standards.